

Lab #5: digital basics

Name: Tony Han

Table partners: Pranay, Theo, Marnu

#1. Experiment description and Prelab

##1.1 Description

In this lab, we will prepare for the final experiment involving a servo motor. I will be familiarizing myself with logic gates, and non-analog circuits. This means that the circuit or circuit parts will have a definitive state (0 or 1) rather than being on a continuum such as current in an inductor.

to do so, I will be using the following:

1. Schmidt trigger logic inverter
2. D latch single memory device
3. DAC based on resistive ladder
4. counter

##1.2 Prelab

Below is my truth table for the latch outcomes with all possible D and E inputs. Note that for the cases where D and E are both off, we have that Q or Qbar can be on while the other is off. This is the same situation for when D is on but E is off. For the two other cases, however, we have definitive outcomes.

This seems quite strange and has no symmetry (ie the cases with definitive outcomes aren't 0,1 and 1,0, but rather 0,1, and 1,1.), but that is likely due to the lack of symmetry in the gate paths too.

Note that: the outcomes for case D(0) E(0) and D(1) E(0) are such that only ONE of Q or Qbar is on due to the nature of NAND gates.

In [21]:

```
InlineImage('newtruth.png', width=12, height=12, caption= "Figure 1, Truth Table for the
```

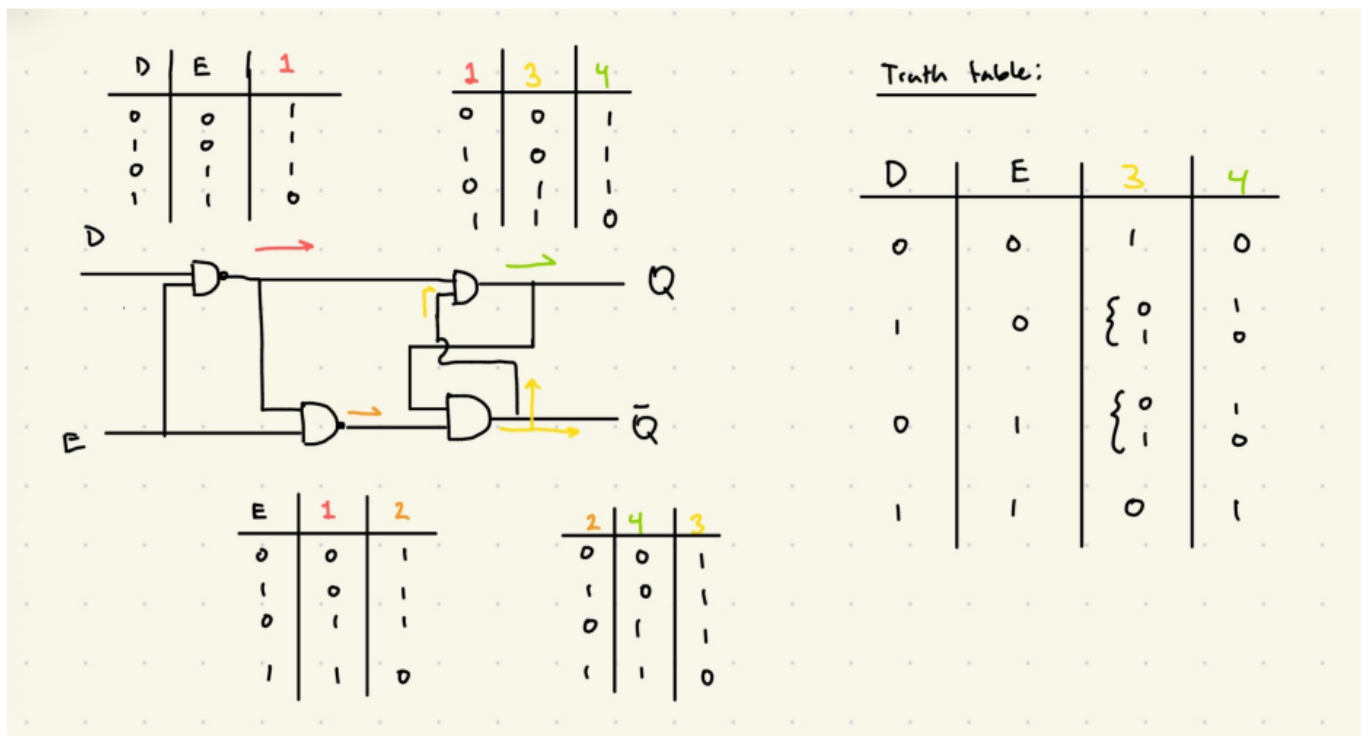


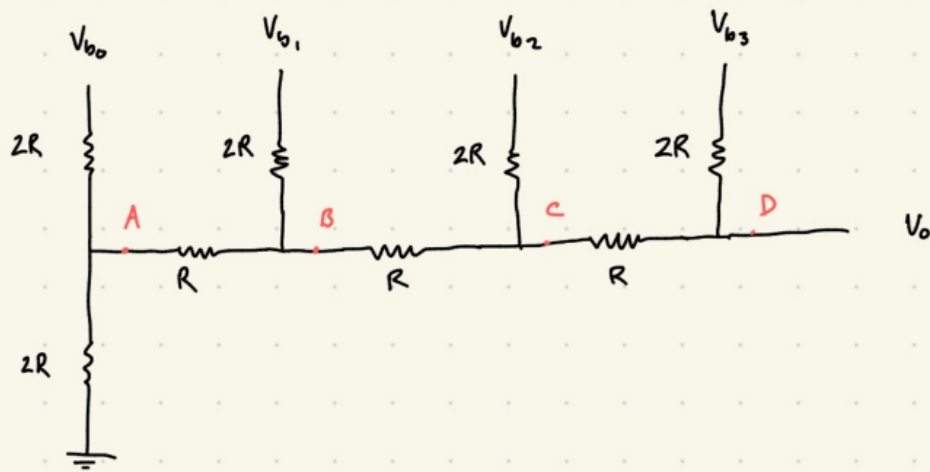
Figure 1, Truth Table for the latch consisting of NAND gates. outputs Qbar and Q are 3 and 4 respectively.

Below is my week 7 prelab.

Here, using thevenin analysis, I found the thevenin resistance to the left of each node for A, B, C, and D.

In [6]:

InlineImage('thevenin.png', width=12, height=12, caption= "Figure 2, thevenin analysis f



A:

$$R_{th} = 2R \parallel 2R = R$$

B:

$$R_{th} = (2R \parallel 2R + R) \parallel 2R = R$$

C:

$$R_{th} = ((2R \parallel 2R + R) \parallel 2R + R) \parallel 2R = R$$

D:

$$R_{th} = (((2R \parallel 2R + R) \parallel 2R + R) \parallel 2R + R) \parallel 2R = R$$

Figure 2, thevenin analysis for a 4 step resistive ladder at the 4 nodes.

Essentially, each node sees a thevenin equivalent resistance of R to the left. Below, I include analysis of what this actually means for any inputs at V_{b0} , V_{b1} , etc.

In [8]:

InlineImage('equivalent.png', width=12, height=12, caption= "Figure 3, input analysis fo

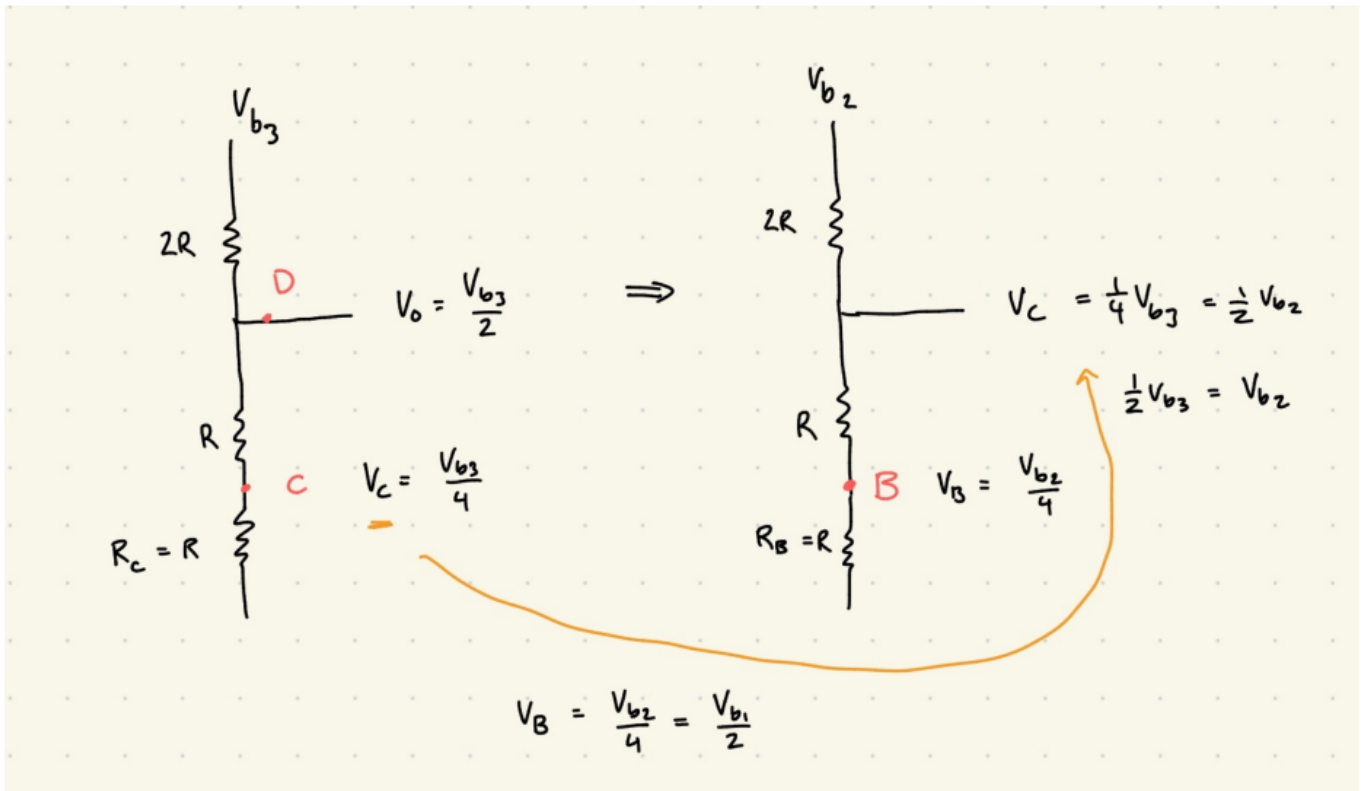


Figure 3, input analysis for voltage drops for a voltage input at a given source.

Basically, what my diagram shows is the voltage drop. I started by illustrating an input from V_{b_3} and what the voltage drop would look like at each relevant node. we can see that the total resistance if seen in series is $4R$, meaning V_o is half of the input and V_c is a quarter. So far, this functions as a simple voltage divider. Then, I looked at the adjacent input to the left, V_{b_2} . Here, we can see the same $4R$ in series, with V_c being half of the input. However, we can also see that this must also be a quarter of the input from the source on V_{b_2} 's right hand side.

so by propogating this behaviour to the other inputs, we know that, starting from V_{b_o} , as we advance up one step, the voltage seen at the next node is half. So for 4 steps like what V_{b_o} travels, V_o sees a voltage of $1/16$ of the input. The denominator is equal to 2 to the power of however many 'steps' it passes to reach the output.

so in essence:

$$V_o \text{ sees: } \frac{V_{b_o}}{16}, \frac{V_{b_1}}{8}, \frac{V_{b_2}}{4}, \text{ and } \frac{V_{b_3}}{2}$$

This gives our desired relationship:

In [9]:

InlineImage('equation.png', width=12, height=12, caption= "Figure 4, desired equation fr

$$V_o = \frac{V_{b0}}{16} + \frac{V_{b1}}{8} + \frac{V_{b2}}{4} + \frac{V_{b3}}{2} \quad (1)$$

Figure 4, desired equation from prelab.

How does this act as a digital to analog converter? Well, we can think of each input as a bit. Since each input is 2x of the voltage input to the left of itself, we can use this behaviour to represent the same type of scaling that bits use (either 0 or 1). An analogy for base 10 counting would be that each bit represents a digit place; ones, tens, hundreds, thousands (left to right). as soon as each one gets hits its max, it resets but the one to the right turns on. Basically, this DAC is a 4 bit system, with the least significant bit on the left and the most significant bit on the right.

For example; the first input can switch states twice (on off, on off) and the second input can switch on to represent two switches of the first bit (base 2). This can repeat 3 times, as we essentially have a 4 bit system. The voltage output is the sum of all of these bits. the least significant bit contributes the least to the output, while the most significant can change the output by a large amount.

We could have a voltage scale where we divide the output by the value of the input of the least significant bit, where the resulting number (should be an integer) is the number of times of an action counted by this DAC.

#2. Experiment

Pre-experiment notes:

- only test inverter to 3.3v
- ground unused inverters
- test ALL D latch states

##2.2 Tasks

###2.2.1 Determining the input voltage to trigger a change in the output of the 74HC14A inverter.

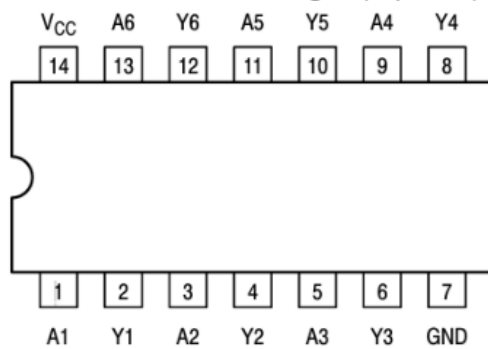
Below is the datasheet for my part. I have grounded all of my unused inverters. I am using a 3.3V DC source connected to V_{CC} to power the inverter, and a triangle wave of 0-3.3V connected to the input. I have one set of scopes measuring the 3.3V triangle wave, and another set measuring the output.

In [35]:

```
InlineImage('datasheet.png', width=12, height=12, caption= "Figure 5, Data sheet for the
```

74HC14

Pinout: 14-Lead Packages (Top View)



FUNCTION TABLE

Inputs	Outputs
A	Y
L	H
H	L

LOGIC DIAGRAM

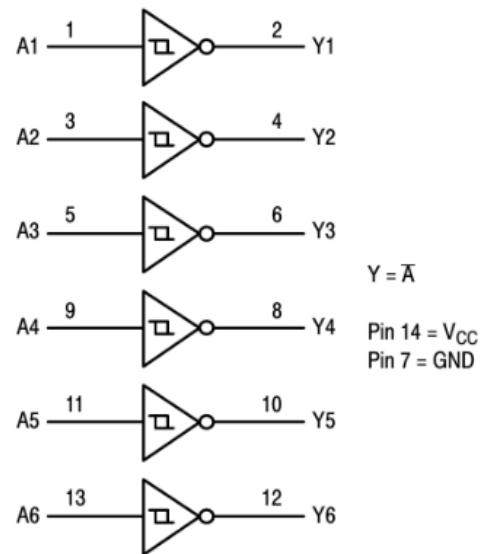


Figure 5, Data sheet for the 74HC14A inverter, sourced from MOUSER electronics.

Here is my inverter setup, along with my circuit diagram:

In [36]:

```
InlineImage('inverterCircuit.png', width=12, height=12, caption= "Figure 6, circuit setu
InlineImage('inverterdiagram.png', width=12, height=12, caption= "Figure 7, circuit diag
```

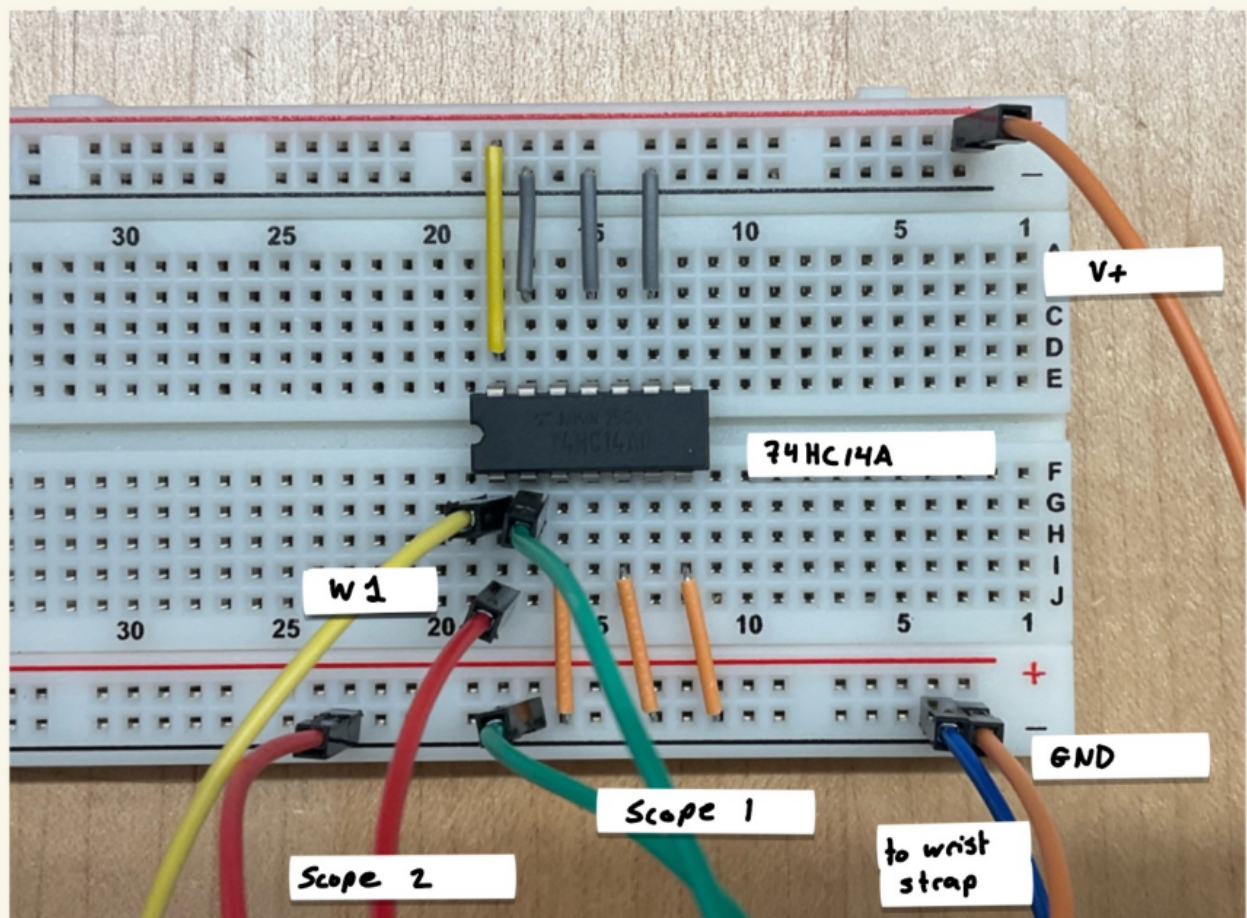


Figure 6, circuit setup for my inverter. Driven by triangle waveform.

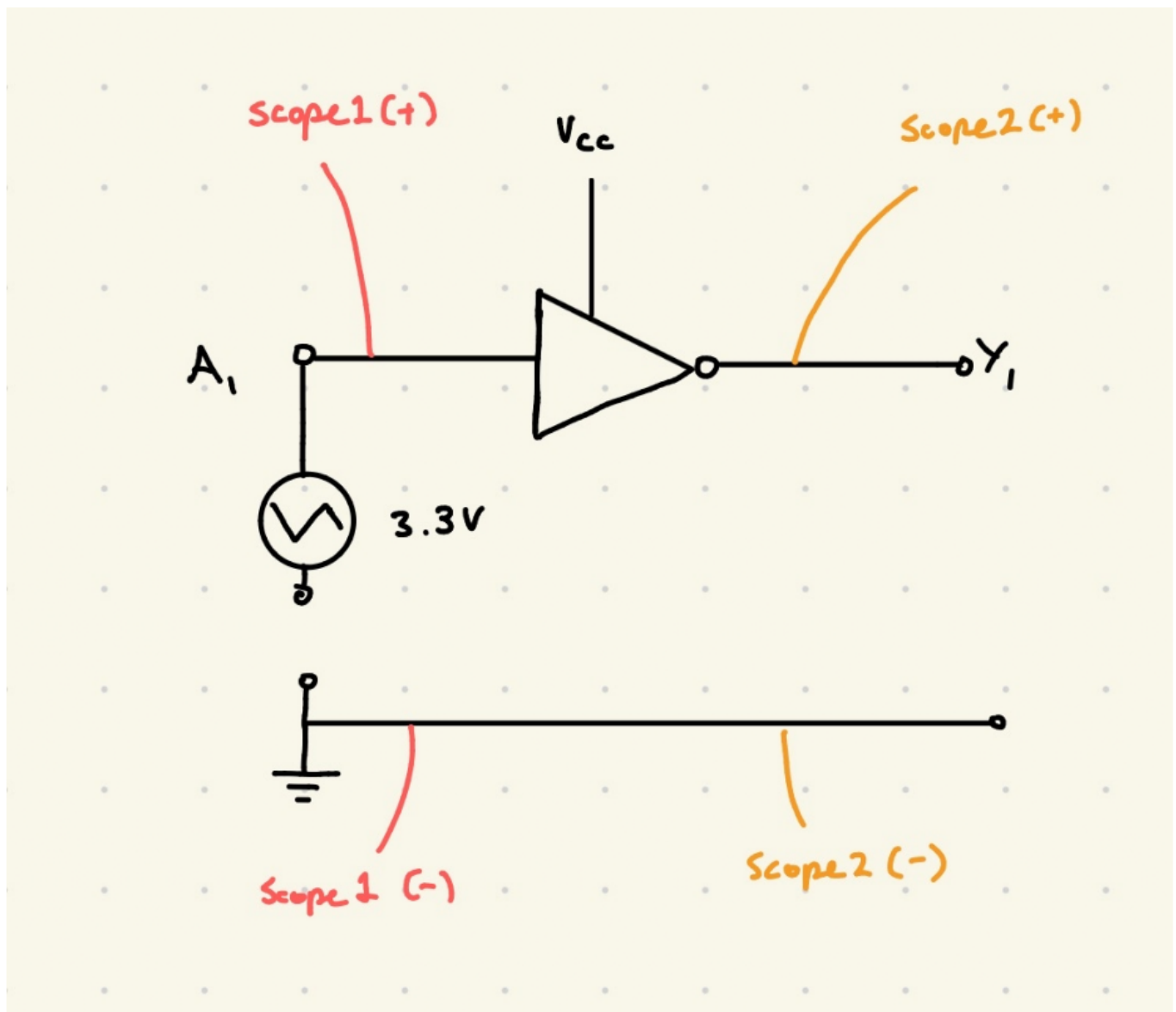


Figure 7, circuit diagram for my inverter. I have simplified it to be only the inverter I am using for clarity.

TS Initially, I had issues with the triangle waveform input. However, I found that this was due to the fact that I had the triangle wave set such that the maximum value exceeded the value of my DC power source for the inverter, so the inverter was truncating the rest of the voltage. This was solved after correcting the triangle wave settings. Below is an image of my initial truncated waveform, along with my new settings to correct it:

In [37]:

```
InlineImage('square.png', width=12, height=12, caption= "Figure 8, truncated triangle wa
```

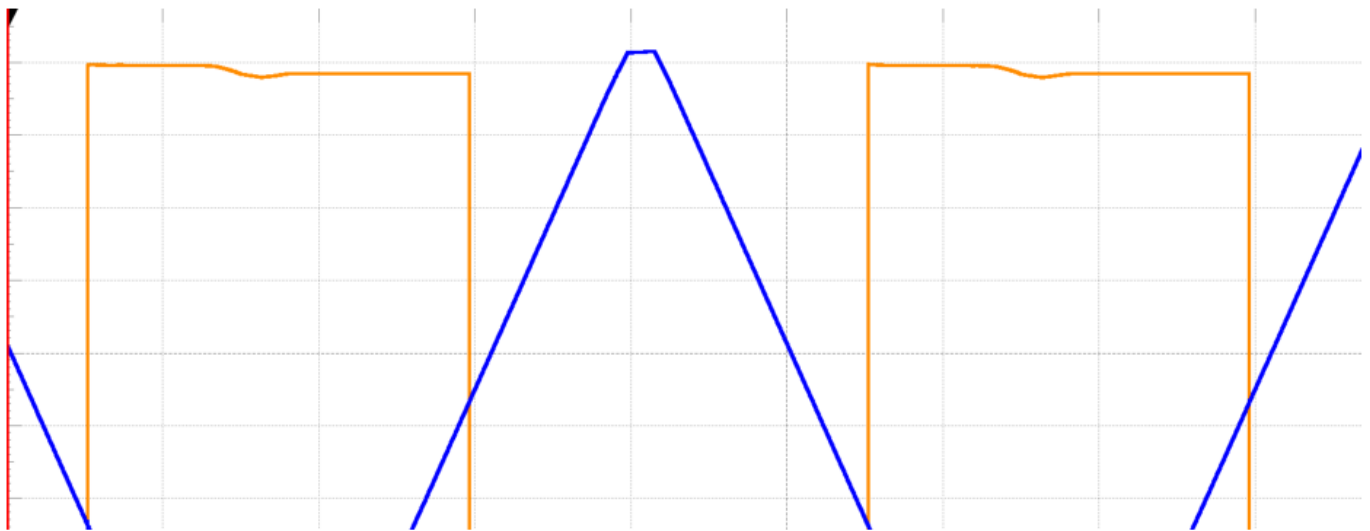


Figure 8, truncated triangle wave with wrong settings.

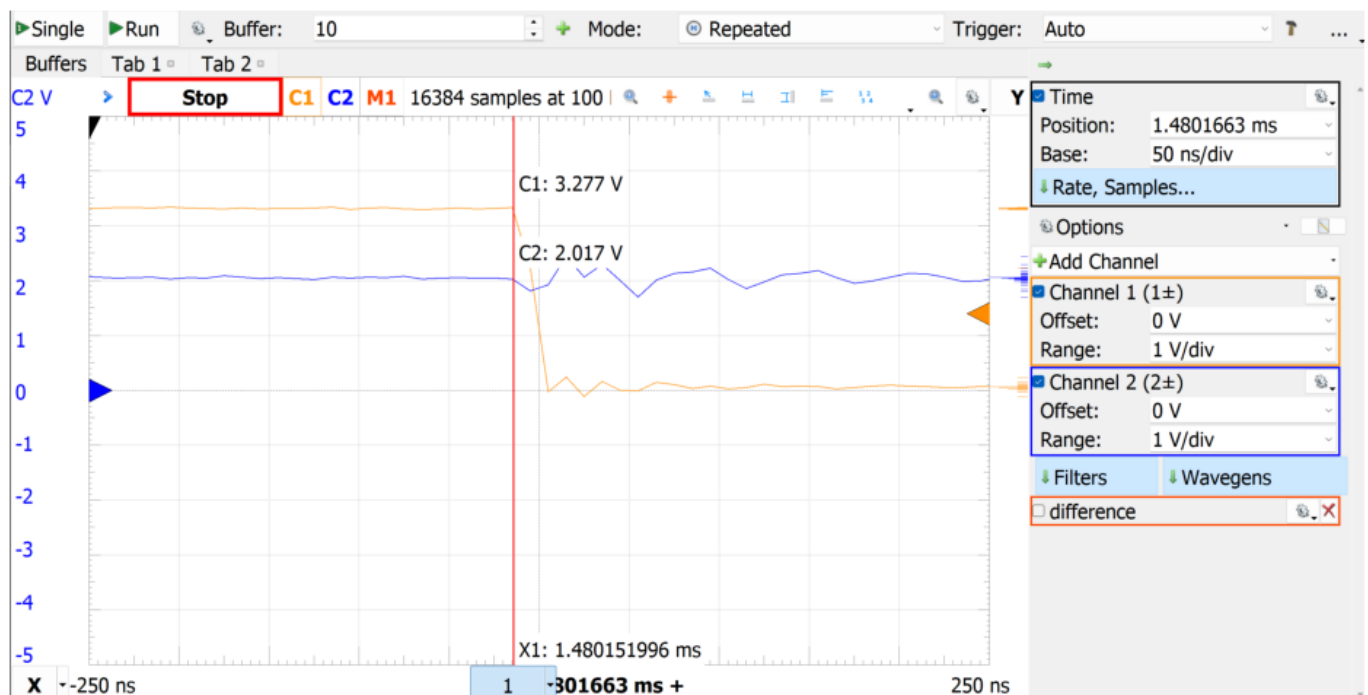
Here is a diagram of my inverter connections, circuit diagram, and pin (i will insert later for the sake of moving on to the D-latch)

In [38]:

```
InlineImage('triggerout.png', width=12, height=12, caption= "Figure 9, waveform of the i
InlineImage('closeuptriggervalue.png', width=12, height=12, caption= "Figure 10, Close u
```



Figure 9, waveform of the input and output voltages over the inverter.
Blue is the input, Orange is the outut.



In [39]:

```
InlineImage('closeuptrigger2.png', width=12, height=12, caption= "Figure 11, close up of
```



As we can see, once the input increases up to 2V, the inverter inverts the signal and outputs 0V. Once the input decreases back down to 1V, the inverter inverts the signal and outputs 1V.

Using the graphing tool in waveforms, I obtained the following hysteresis plot for the inverter. In this case, I think the graph looks like this because the voltage threshold to trigger a rise is different than that to trigger a drop. Hence, when it is plotted against time with a triangle source wave, they will not drop 'in phase' with each other. They are slightly offset, which causes this hysteresis visual.

In [40]:

InlineImage('hysteresis.png', width=12, height=12, caption= "Figure 12, hysteresis effect of inverter voltage, caused by different threshold voltages for on/off transitions..")

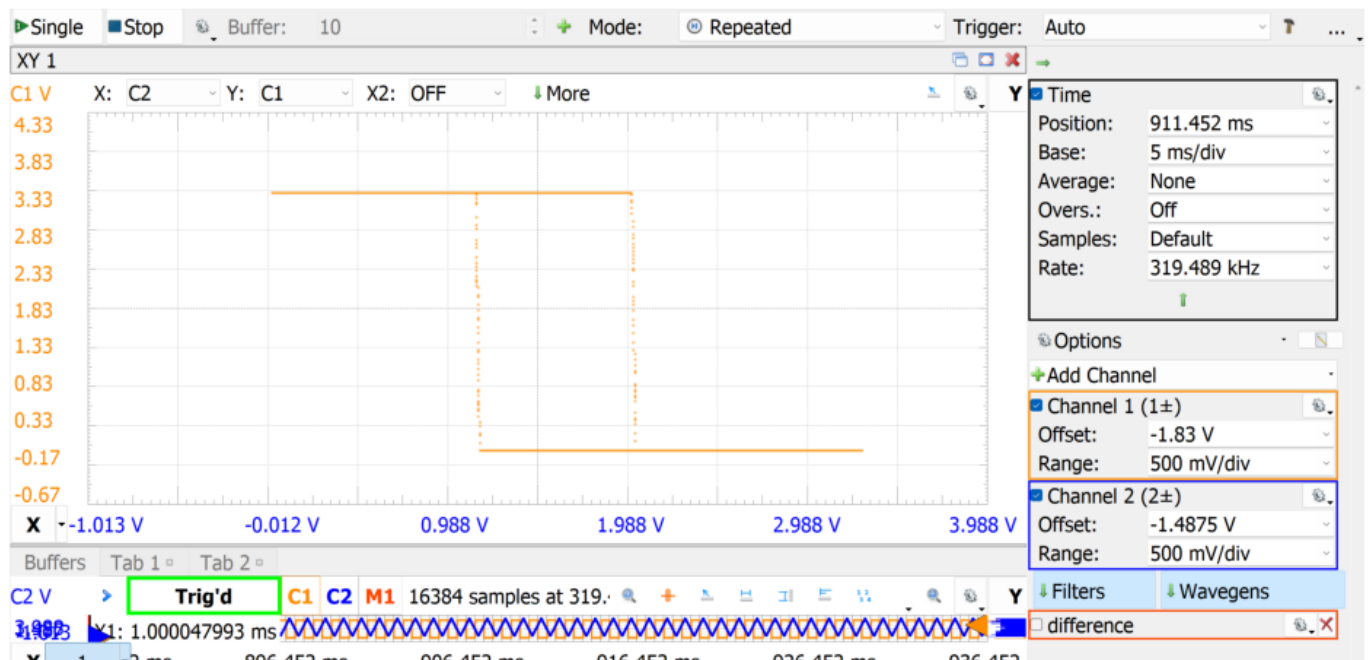


Figure 12, hysteresis effect of inverter voltage, caused by different threshold voltages for on/off transitions..

An inverter offers noise immunity because it can only output two states. These states are either high or low, with a large amount of separation in magnitude between the two. Hence, any minor fluctuations or external perturbation is not enough to make the output reading ambiguous.

Furthermore, the input also has a certain threshold to get the inverter to trigger. So; if an inverter triggers once, it MUST wait until the threshold to switch to its next state is met. So any noise won't cause unintended inverter behaviours. Unless, of course, the noise is somehow large enough to trigger the inverter in its fluctuation (ie have a range of like 3 volts or something). But at that point, there are other major problems with your circuit lol.

It can also square up signals with long rise/fall times due to its nature of switching states pretty much instantly once a specific threshold is hit. As we can see in the hysteresis diagram, despite the fact that the inverter's input is a triangle wave, we see a sharp rise and drop in voltage; as soon as the threshold is hit, the inverter triggers. The inverter also only outputs high or low, meaning the values it goes to are the same provided the range/amplitude of the voltage input does not fluctuate.

###2.2.2 D latch

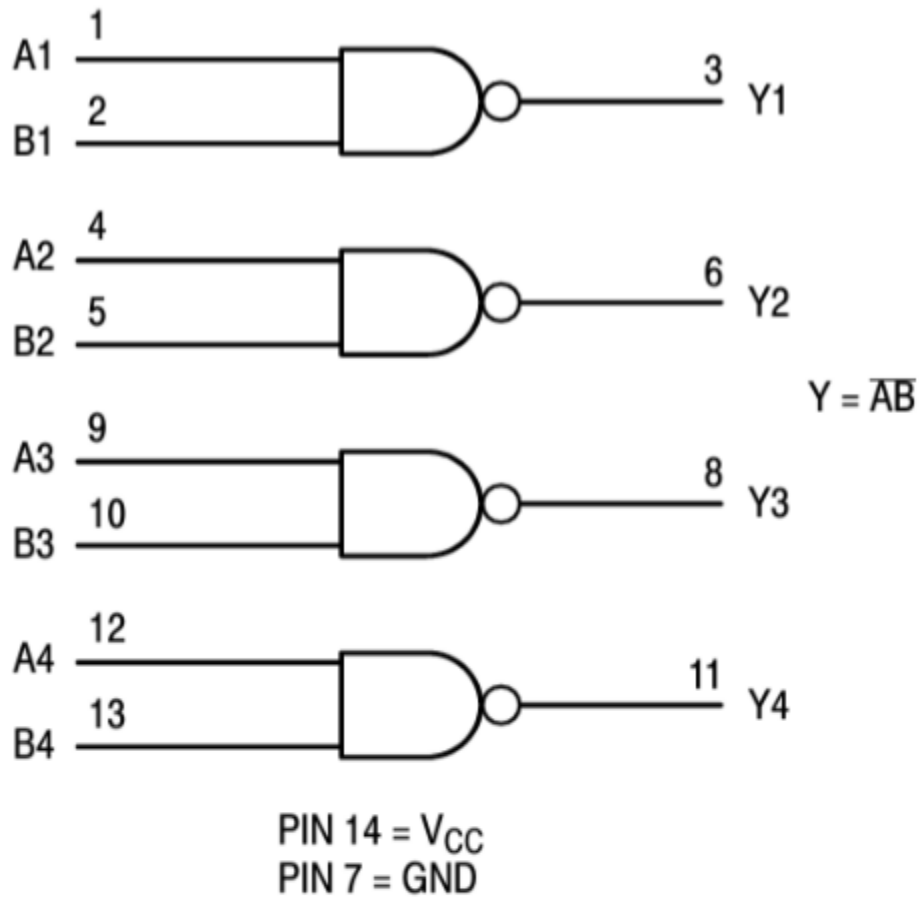
First, I will confirm the operation of a single NAND gate, confirming its truth table using the logic and patterns app in waveforms:

First, I set up a nand gate according to the datasheet. I have included a screenshot of the datasheet schematic below.

In [41]:

InlineImage('nanddata.png', width=12, height=12, caption= "Figure 13, data sheet for the")

LOGIC DIAGRAM



Pinout: 14-Lead Packages (Top View)

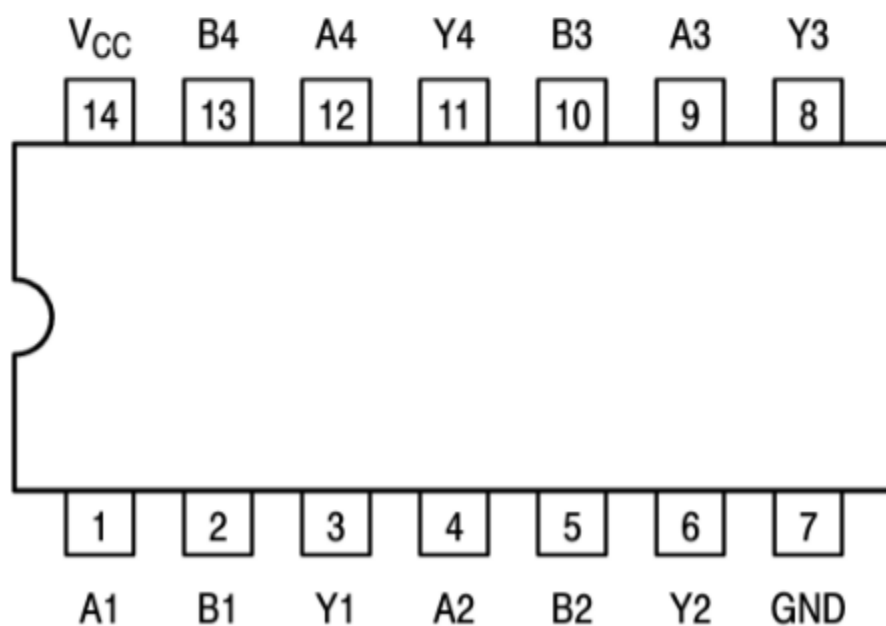


Figure 13, data sheet for the NAND gate.

In lieu of an annotated photo of my setup, I will give a brief description so I can move on to the more important part of the lab.

I connected Vcc to a 3.3V power supply to power the circuit part. Then, I connected 2 DIO pins from my ad3, one to A1 and one to B1. These two act as inputs. I connected a third DIO pin to Y1 to read the output.

I set pin A1 to oscillate with twice the frequency that B1 oscillates at, so I could observe all of the output waveforms in the LOGIC app. Doing so, I obtained the following truth table:

A	B	Output (Y)
0	0	1
0	1	1
1	0	1
1	1	0

This matches the theoretical/expected behaviour of a single NAND gate, where the output is high in all cases apart from both inputs low.

Then, I will build the D-latch:

In [42]:

```
##INCLUDE THE D LATCH SCHEMATIC, SETUP, AND CIRCUIT DIAGRAM!  
InlineImage('dlatcheasy.png', width=12, height=12, caption= "Figure 14, photo showing d
```

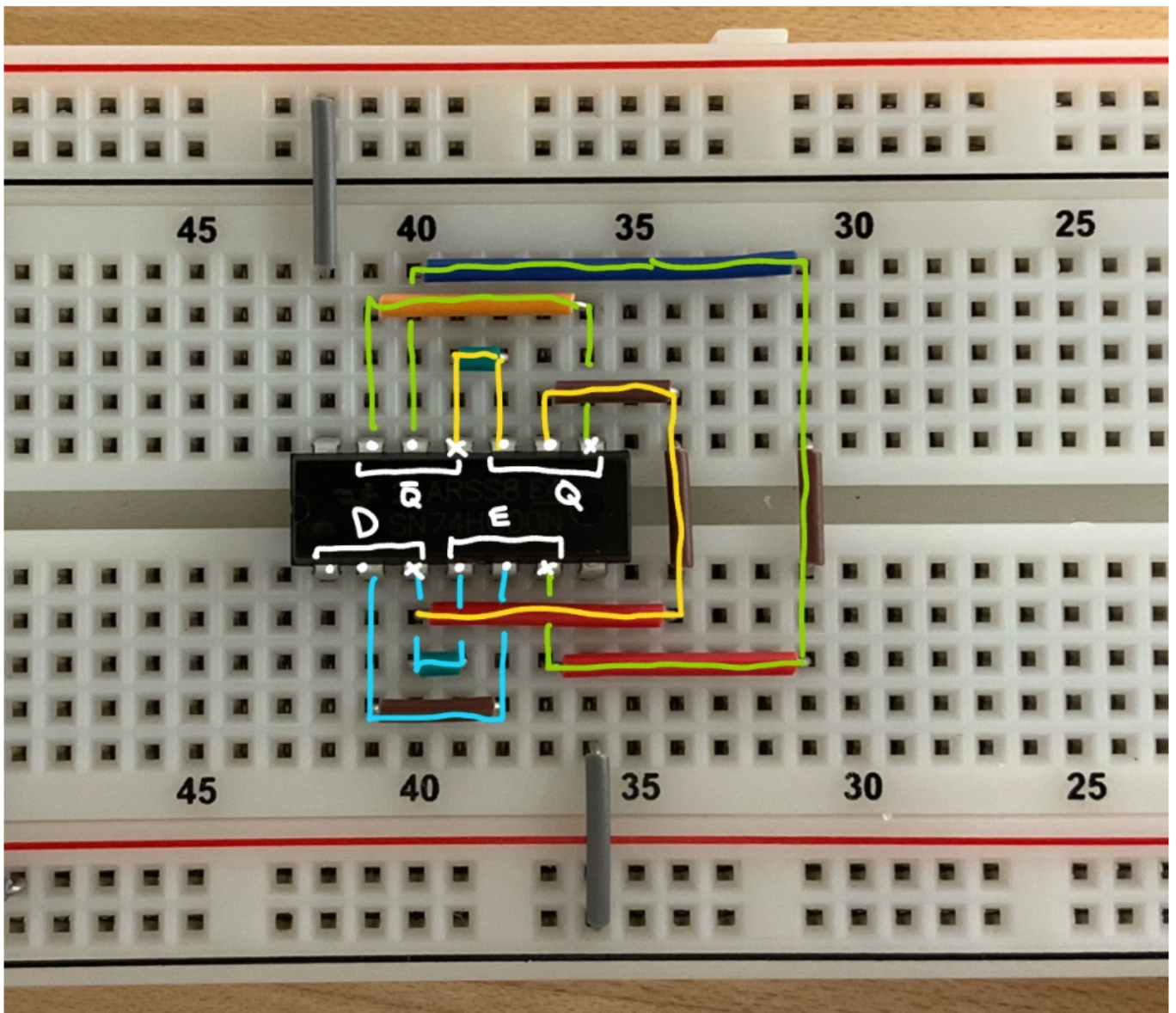


Figure 14, photo showing d latch circuit connections ONLY. this is for me to rebuild the circuit later if necessary, and debug.

In [43]:

```
InlineImage('realDlatch.png', width=12, height=12, caption= "Figure 15, actual annotated
```

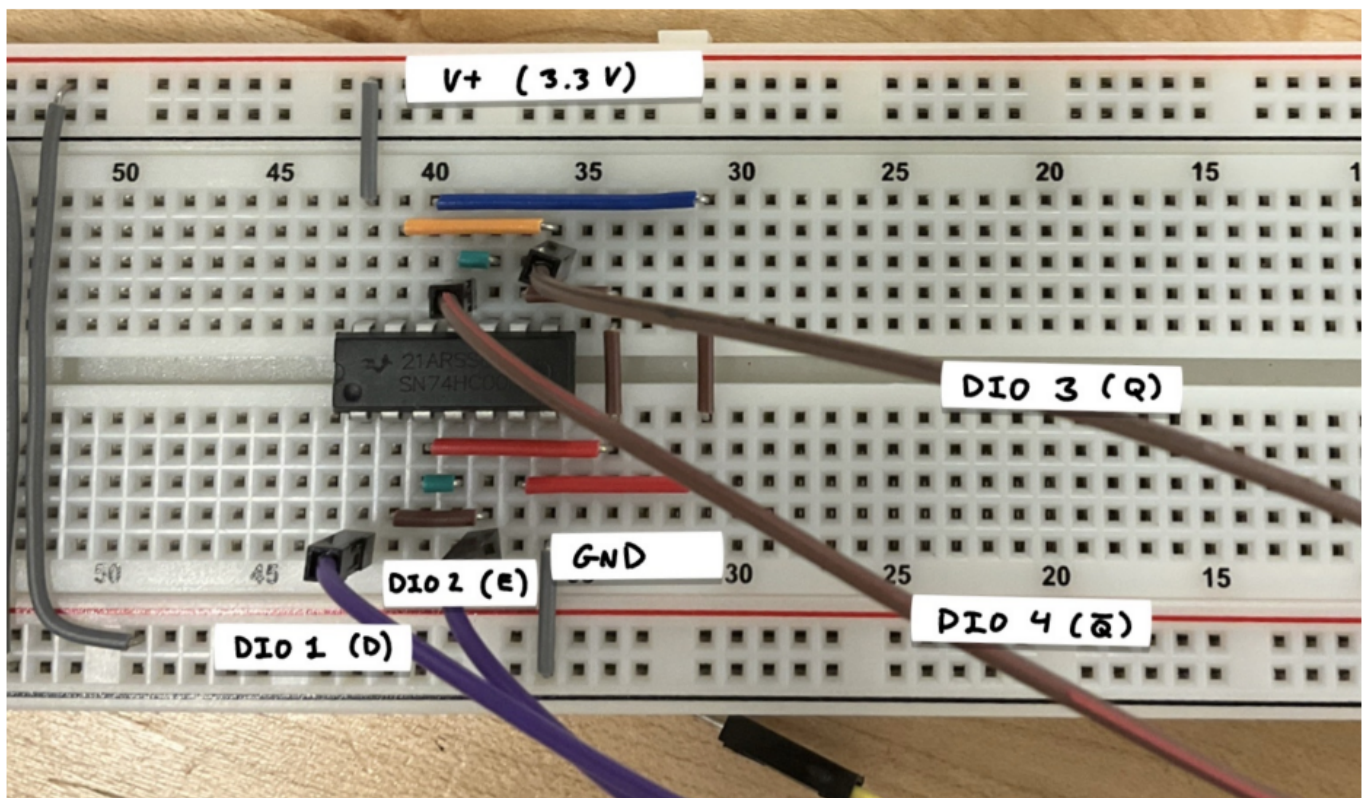


Figure 15, actual annotated photo of my D latch circuit setup.

In [44]:

```
InlineImage('dlatchdiagram.png', width=12, height=12, caption= "Figure 16, circuit schem
```

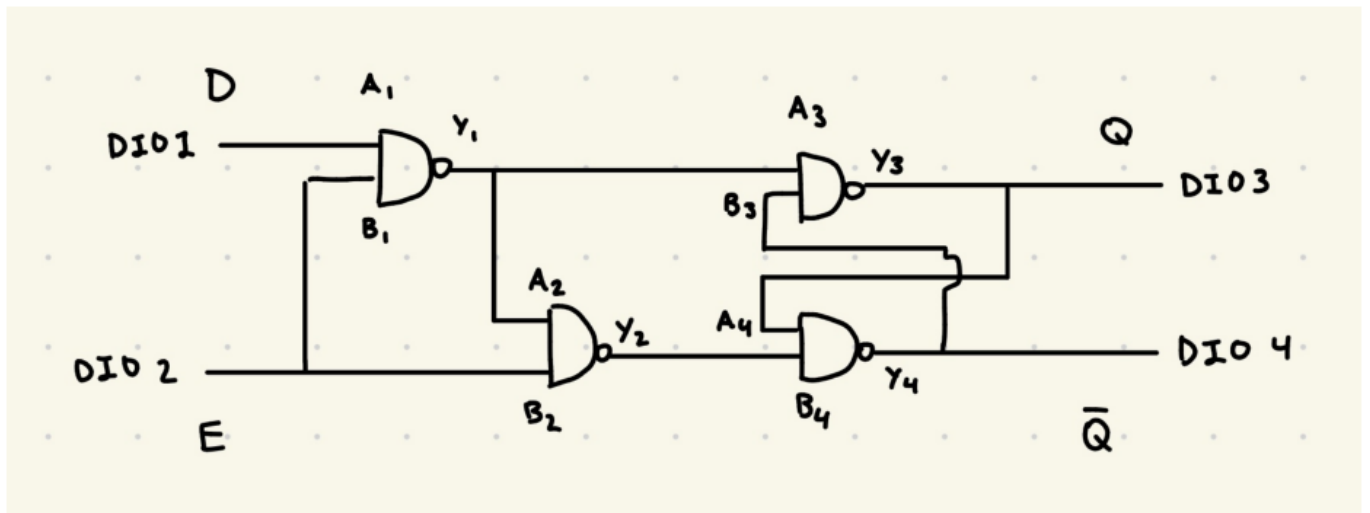


Figure 16, circuit schematic diagram for my D-latch setup.

for D latch, i set the frequency of the digital inputs such that the ratio between them was 5/3. This was done to capture every possible outcome for the D latch. Basically, if the frequencies are some nice integer multiple of each other, I will fail to observe specific cases.

In [45]:

```
##INCLUDE THE SCREENSHOT OF THE D LATCH WAVEFORMS FROM AD3 LOGIC.
```

```
InlineImage('dlatch.png', width=12, height=12, caption= "Figure 17, d-latch digital outp
```

```
InlineImage('dlatchsettings.png', width=12, height=12, caption= "Figure 18, D latch inpu
```

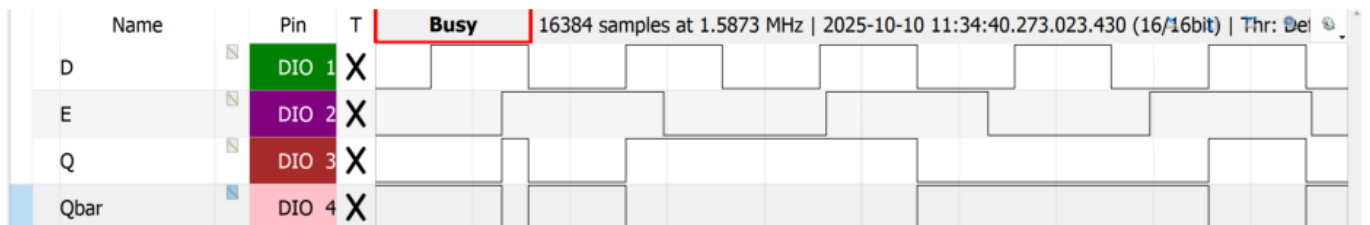


Figure 17, d-latch digital output readings on AD3.

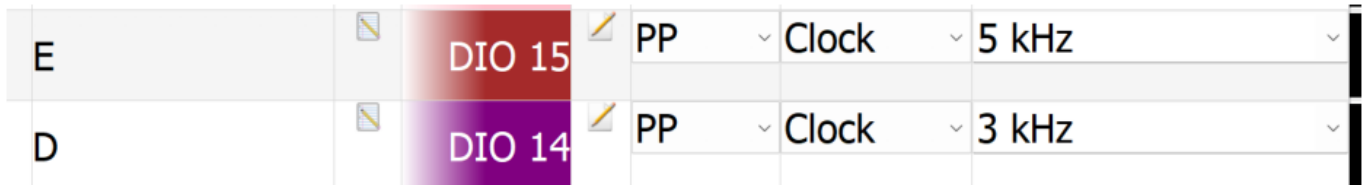


Figure 18, D latch inputs driving at different frequencies.

NOTE: I forgot to take a screenshot of the input settings during the lab. I recreated it after, using different DIO pins. Hence the different DIO pins.

For my outputs/truth table:

D	E	Q_prev	Q	$\bar{Q}$
0	0	0	0	1
0	0	1	1	0
1	0	0	0	1
1	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	1	0	1	0
1	1	1	1	0

How does the D latch store a bit of information? I will reason this out by examining the Q prev column. I am not quite sure what I am looking for right now. This column seems to be important as it affects what the values will be for Q and Qbar after some change in D and/or E...

After some thinking, I noticed a pattern. Whenever E is 0, Q is the same as Qprev. I think that because of this, we can use the input of E to store some value so that even if D changes, as long as E is still 0, Q will read its previous value. Because of this, regardless of D, if E is 0, Q will be held at either 1 or 0 without changing due to D. Hence the 1 bit of information being stored. It is trapped in the value of Q.

So maybe E = high is when it can store info, and E = low is when the info is stored/trapped.

Why does this actually happen? I think this happens because of that feedback from the outputs of Q and Qbar, acting as a reinforcing loop almost. It ensures that the value is held inside the D latch, since an output from one NAND goes back into the input of the other NAND gate. And also, Q and Qbar must be opposite states as each other.

###2.2.3. DAC

First, I set up the circuit following the instructions of the data sheet, with a digital input at each node of the R-2R dac:

In [46]:

```
InlineImage('datasheet1.png', width=12, height=12, caption= "Figure 19, datasheet for th  
#InlineImage('.png', width=12, height=12, caption= "Figure , my physical R-2R dac setup  
#UPDATE IMAGES FOR THE DAC - NEED SCHEMATIC, PHOTO, AND CIRCUIT DIAGRAM
```

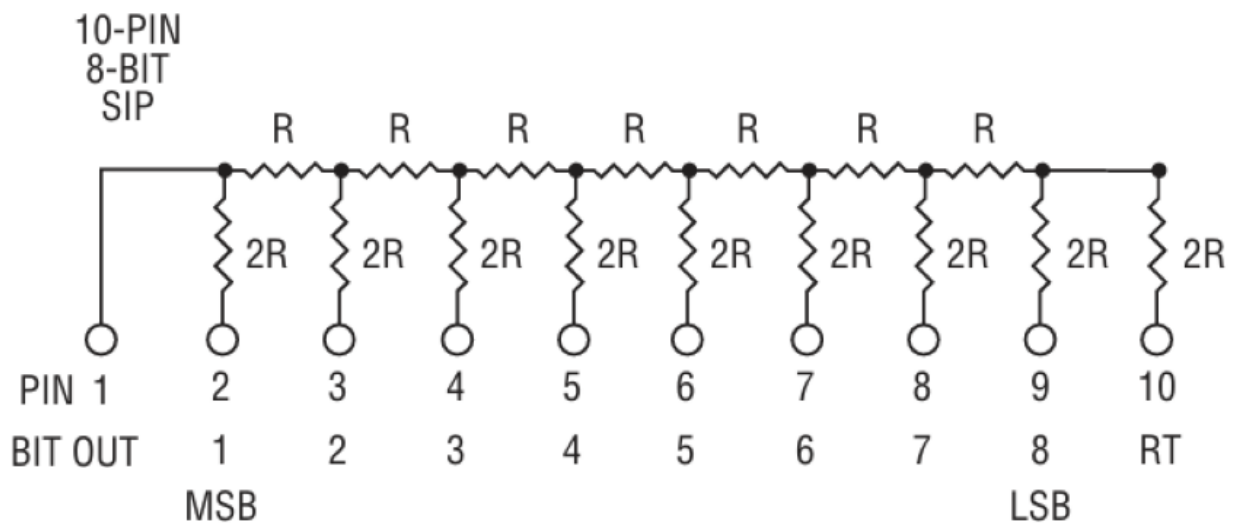


Figure 19, datasheet for the R-2R dac which I used to setup my circuit.

In [47]:

```
InlineImage('dacSchematic.png', width=12, height=12, caption= "Figure 20, schematic of m  
InlineImage('dacdiagram.png', width=12, height=12, caption= "Figure 21, circuit diagram  
#InlineImage('.png', width=12, height=12, caption= "Figure , my physical R-2R dac setup  
#UPDATE IMAGES FOR THE DAC - NEED SCHEMATIC, PHOTO, AND CIRCUIT DIAGRAM  
#InlineImage('.png', width=12, height=12, caption= "Figure , my physical R-2R dac setup  
#UPDATE IMAGES FOR THE DAC - NEED SCHEMATIC, PHOTO, AND CIRCUIT DIAGRAM
```

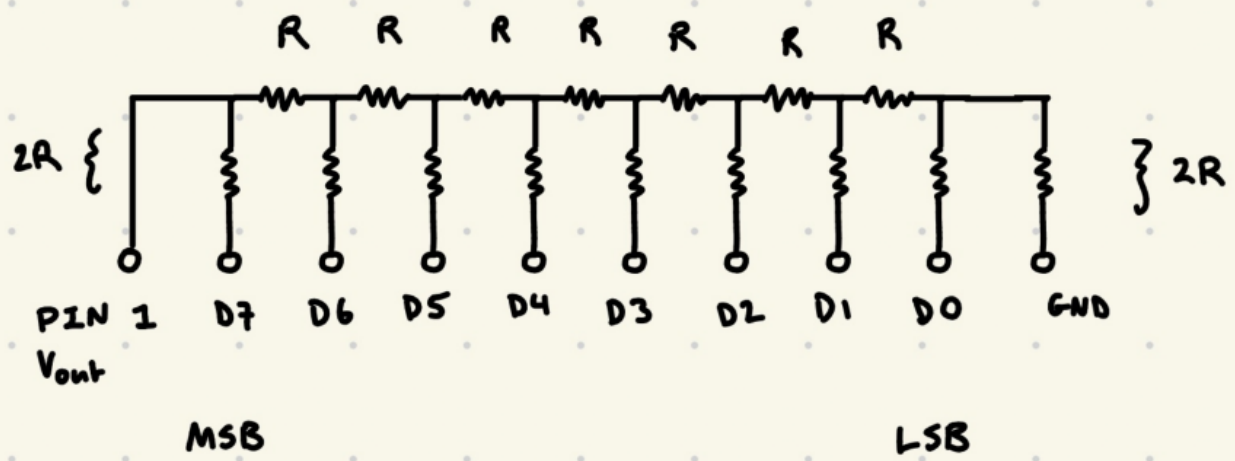


Figure 20, schematic of my own R2R DAC.

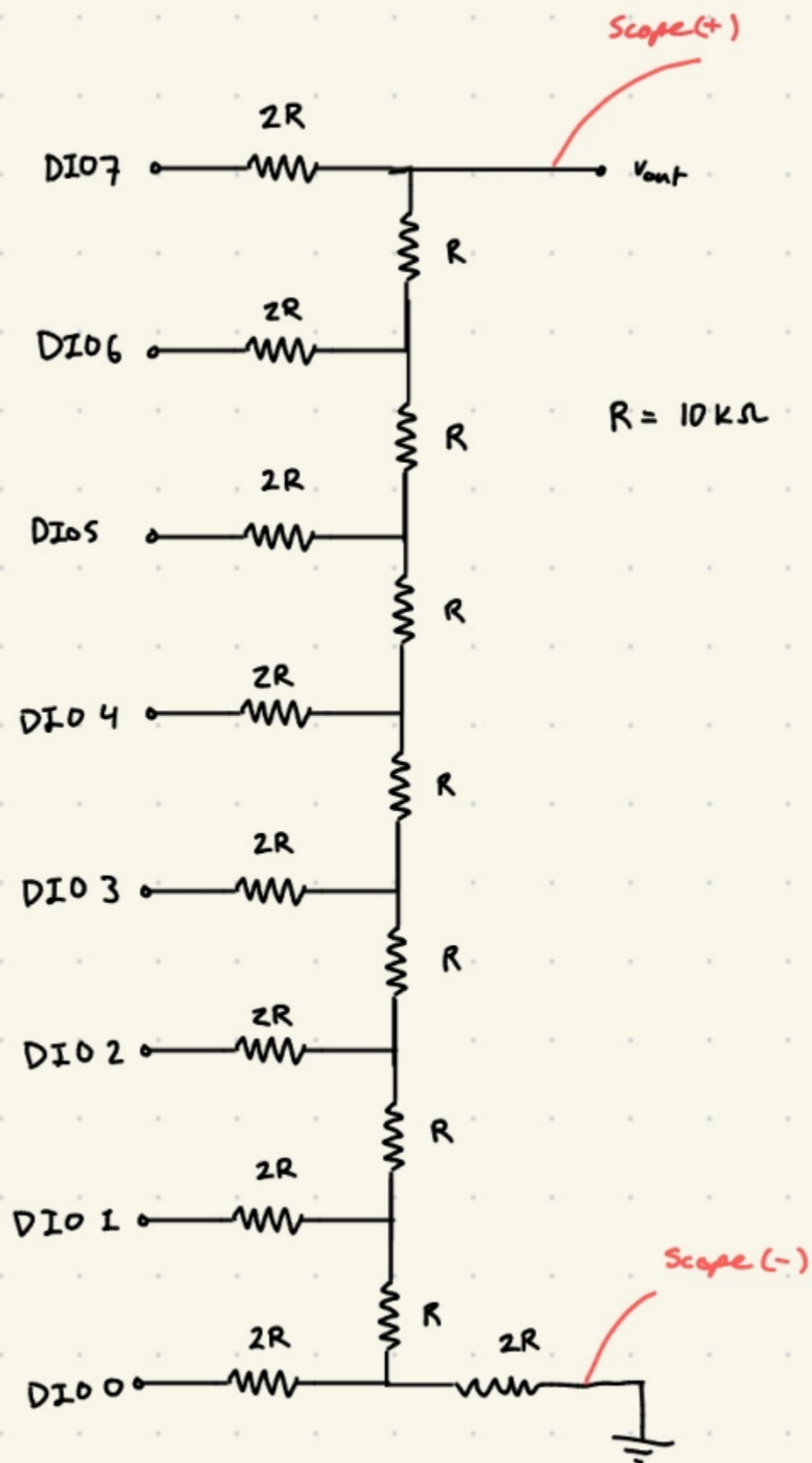


Figure 21, circuit diagram of my R2R DAC setup. Value of R was read from the data sheet.

Here is a photo of my actual setup:

In [48]:

```
InlineImage('dacsetup.png', width=12, height=12, caption= "Figure 22, setup of my R2R DA
```

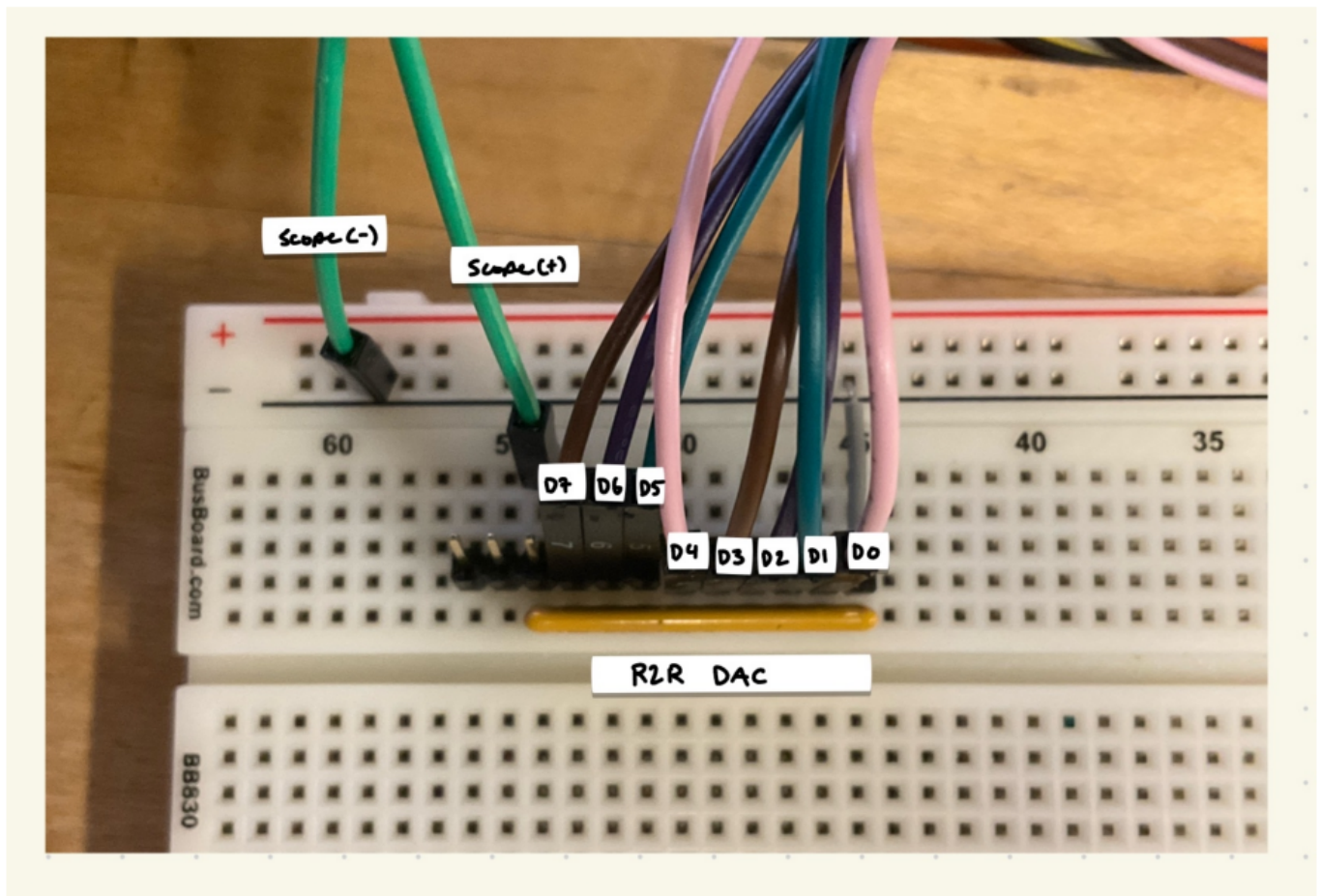


Figure 22, setup of my R2R DAC with 8 DIO pins, and 2 scope probes at output and ground.
D = DIO, D7 is MSB and D0 is LSB

Then, I set the digital inputs like so: my most significant bit D7 operates at 250 Hz, which is the frequency of the sawtooth wave we want to find. My settings are seen below.

In [49]:

```
InlineImage('inputs.png', width=12, height=12, caption= "Figure 23, R-2R digital input s
```

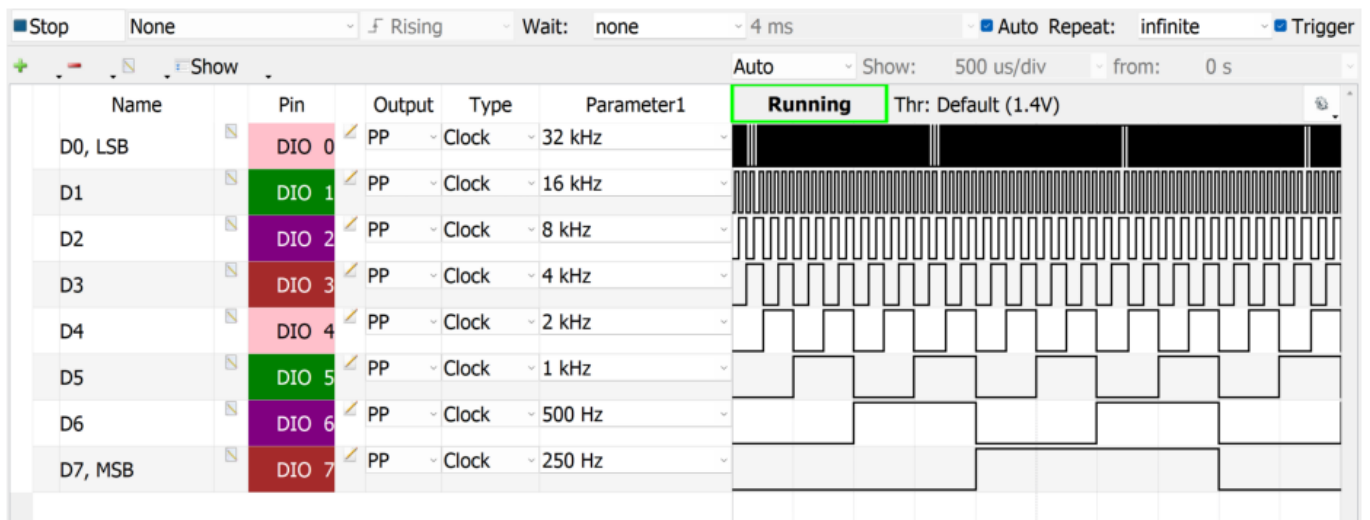
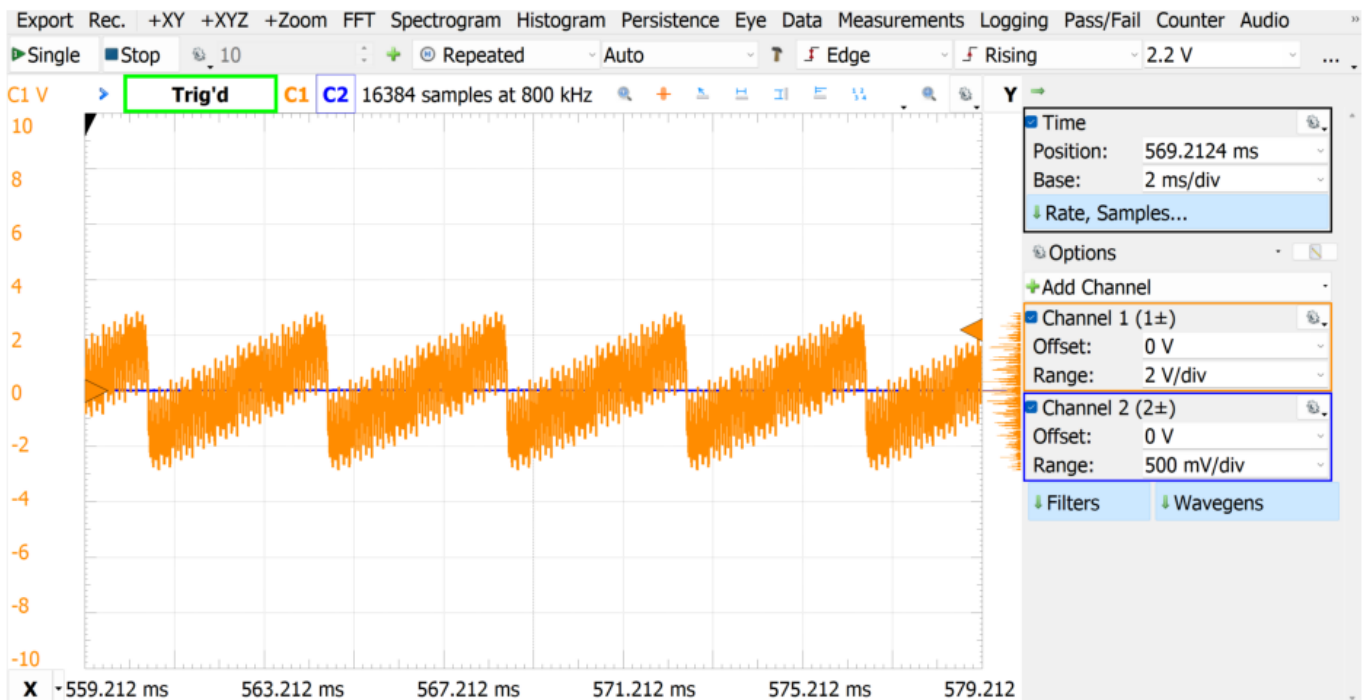



Figure 23, R-2R digital input settings

TS Initially, I see a sawtooth wave but it has a lot of noise. See below. However, I realized that I neglected to connect the ground to the GND of my AD3. Whoops. Now I see the correct sawtooth wave. Below are my initially incorrect, as well as the correct sawtooth waveforms.

In [50]:

```
InlineImage('badsaw.png', width=12, height=12, caption="Figure 24, Initial sawtooth wit  
InlineImage('goodsaw.png', width=12, height=12, caption="Figure 25, final sawtooth of R
```



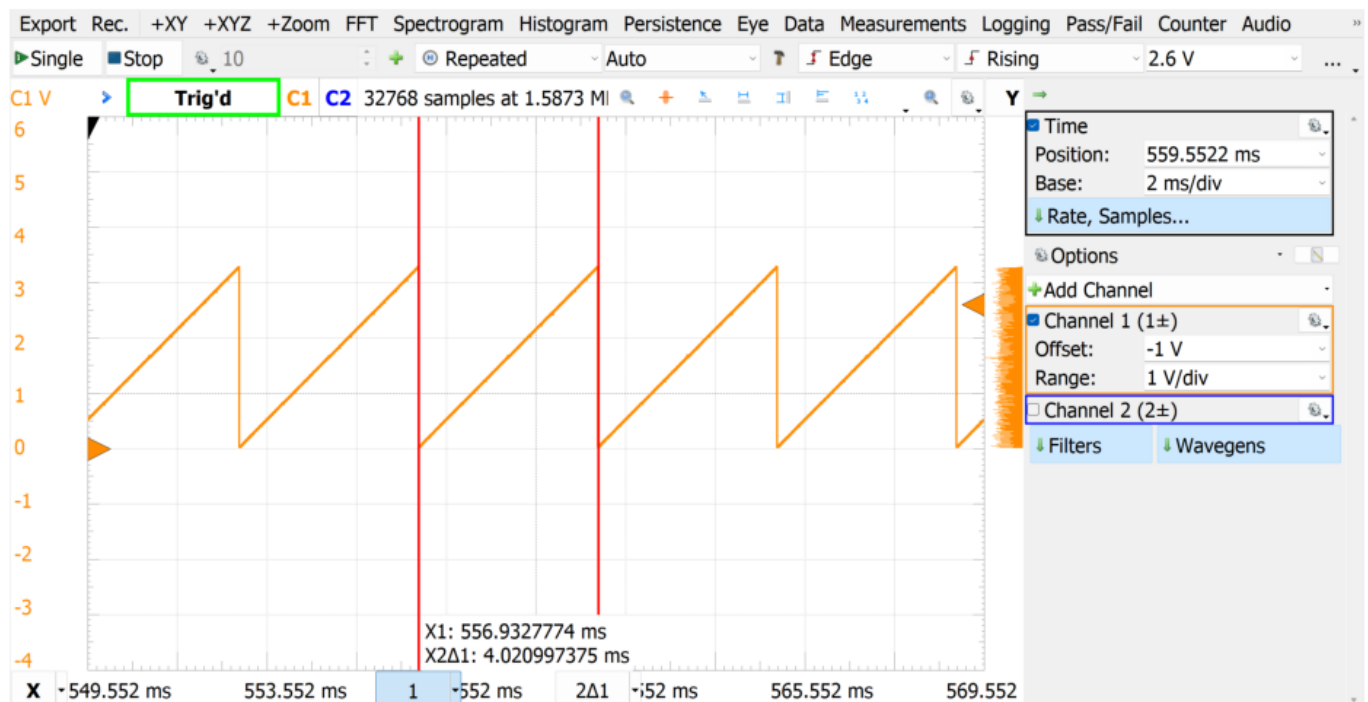


Figure 25, final sawtooth of R-2R dac converting 8 bit digital input to analogue output. Ground connected

As we can see, the period is approx 4ms which corresponds with a 250 Hz frequency.

Now, what is the resolution of the sawtooth? well, we know that the smallest increment in voltage change is what the least significant bit contributes. This bit switches between on and off most frequently, acting as the smallest 'count' for the circuit. This is analogous to counting up by integers in our base 10 number system. counting up by integers means that the smallest possible value we count by is 1; we only increase the tens digit once 1 hits 10.

likewise, we only switch the second bit once the first bit toggles twice (count to 2, since base 2 system). So, we can find the resolution by finding the range of the sawtooth wave, and divide it by how many times the least smallest bit switches in the time the most significant bit switches once, which is $2^8 - 1$ times since we have 8 bits. This is because there are 2^8 different values, but $2^8 - 1$ steps to reach each value. (and a resolution is the smallest step)

Putting this all together, knowing that our digital input from the AD3 operates at a value of 3.3V:

$$\frac{3.3V}{2^8 - 1} = 0.01294...V$$

The THEORETICAL value for resolution is 0.013V, reported to 3 decimal places. I will not evaluate uncertainty this time around as it is not specified in the lab manual.

Now, I will check the actual step size of the waveform by zooming in a lot on my scope.

In [51]:

```
InlineImage('step size.png', width=12, height=12, caption= "Figure 26, magnified image o
```

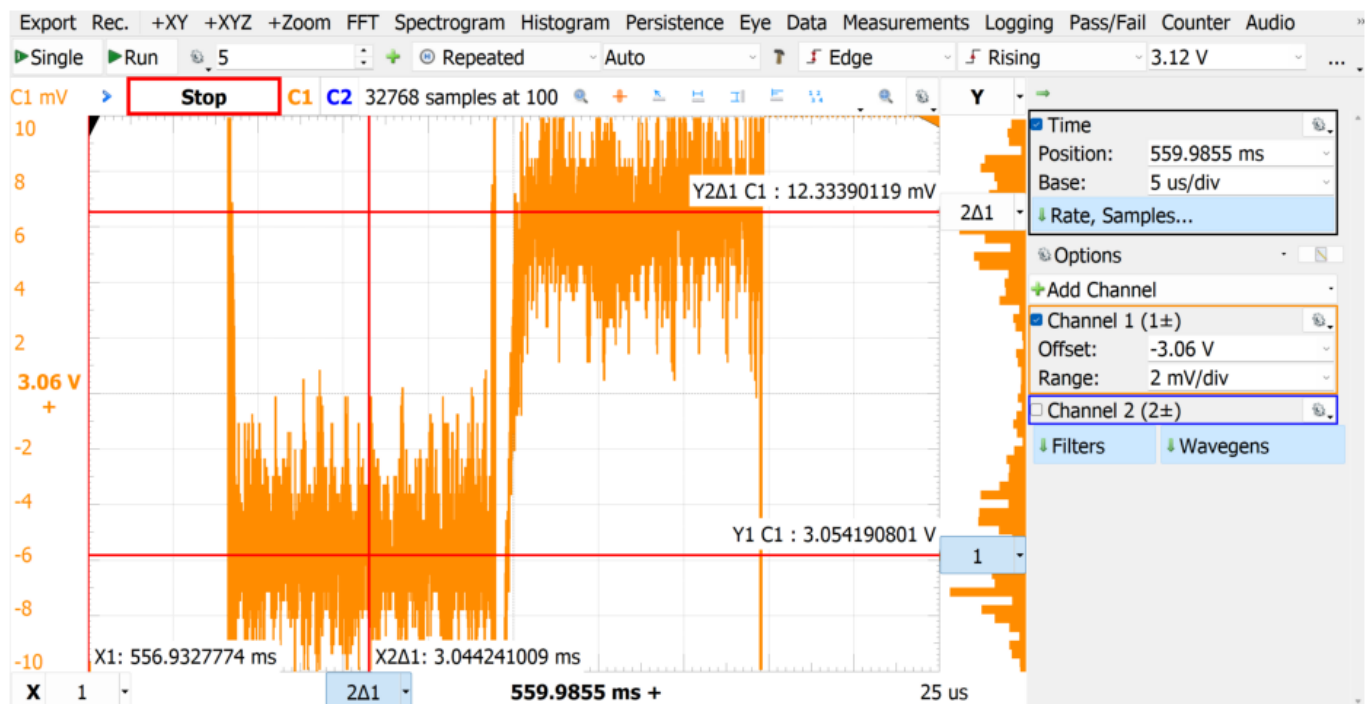


Figure 26, magnified image of sawtooth waveform from the R-2R DAC to find the resolution of the waveform.

as I measured with the cursors, the step size is approximately 12.3mV, which is very close to the theoretical value of 12.9mV.

Now, adding a 1K resistor to simulate a load, I get a similar sawtooth waveform. However, all of the attributes are the same apart from the amplitude of the sawtooth. It now starts from 0 volts and reaches a maximum of 300mV:

In [52]:

```
InlineImage('loadedsaw.png', width=12, height=12, caption= "Figure 27, datasheet for the
```

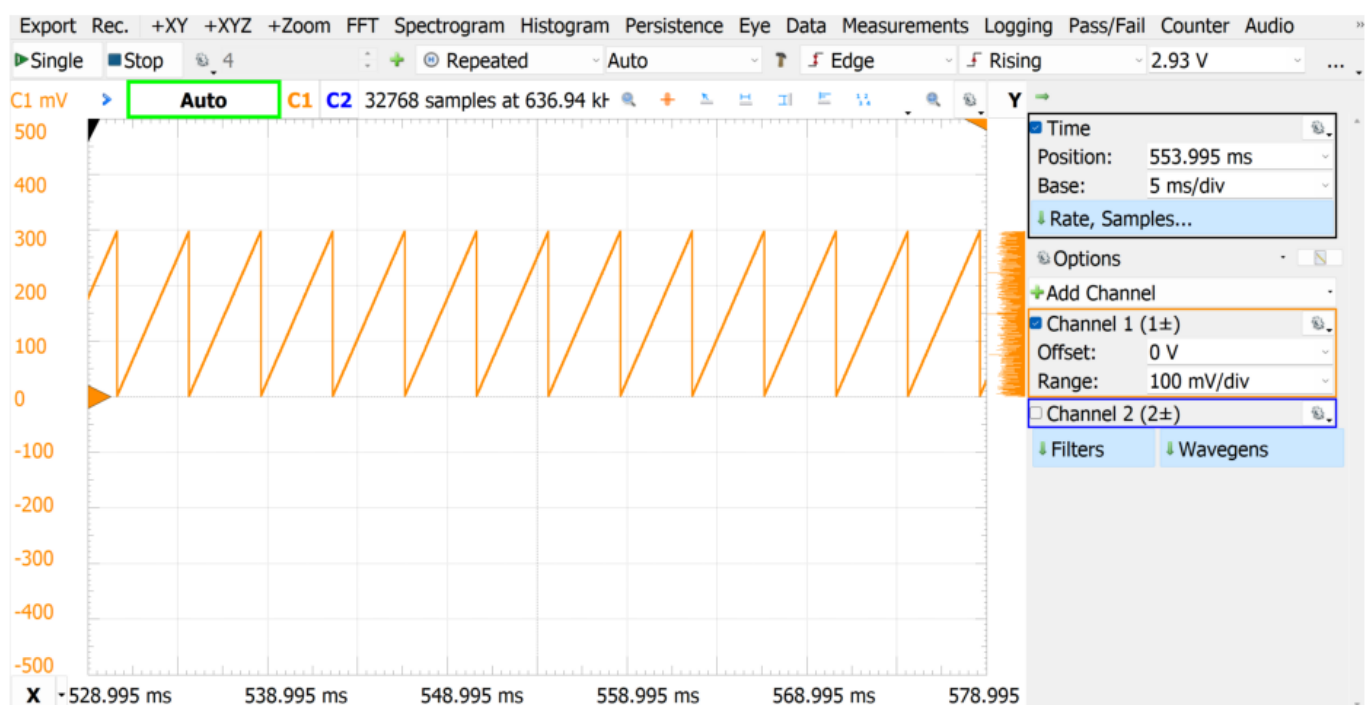


Figure 27, datasheet for the R-2R dac which I used to setup my circuit.

Why does this happen? well, the sawtooth represents counting up. The LSB still triggers the same amount of times in the process of reaching the peak value. However, this peak value is just decreased.

we can explain this by looking at the thevenin equivalent of the R2R dac. From the prelab, I already proved that the thevenin equivalent resistance looking in from any node to the left is R. As such, if we have a load resistor, we have essentially 2 resistors in series; one with the value of the load resistor, the other with the thevenin equivalent resistance of the DAC. In our case, the load resistor is 1k. From the data sheet I know that R in the DAC is 10K. So we have a voltage divider with a 1k and 10k in series.

This logically makes sense; we see a decreased sawtooth amplitude of 300mV over the load, which is exactly the $\frac{1k\Omega}{11k\Omega} * 3.3V = 0.3V$ we expect to see.

##2.2.4 Counter

First, I will test the functionality of the counter and set up a simple divide by 2 counter. I have set it up according to the pin layout from the data sheet:

In [53]:

```
InlineImage('counterdatasheet.png', width=12, height=12, caption= "Figure 28, datasheet  
##insert images for circuit setup, as well as the schematic diagram drawn for it
```

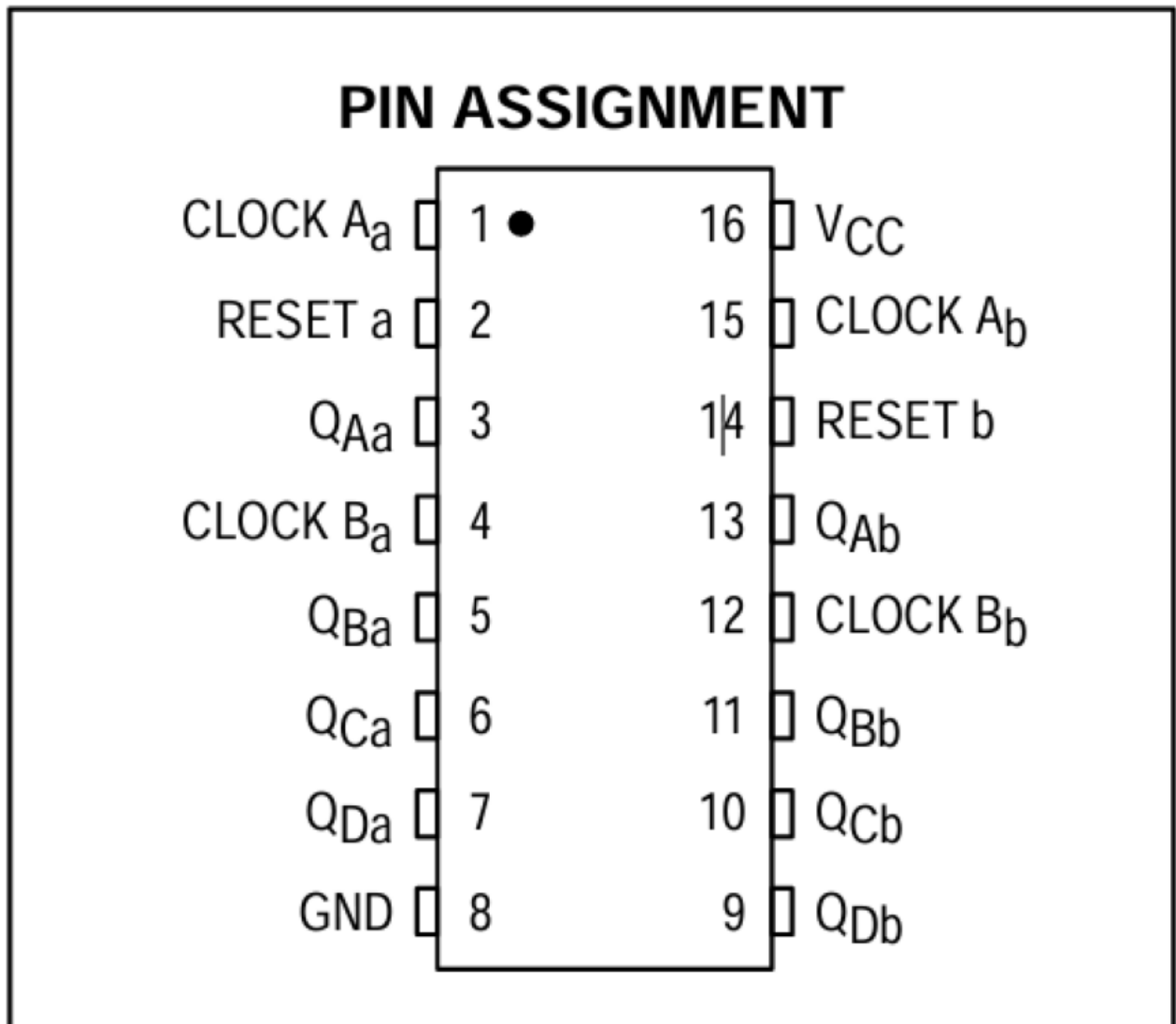


Figure 28, datasheet for the pin layout of the MC54/74HC390 counter

In [54]:

#put the circuit diagram, photo, and schematic here.

InlineImage('divby2photo.png', width=12, height=12, caption= "Figure 29, divide by 2 cir

InlineImage('divby2schema.png', width=12, height=12, caption= "Figure 30, divide by 2 ci

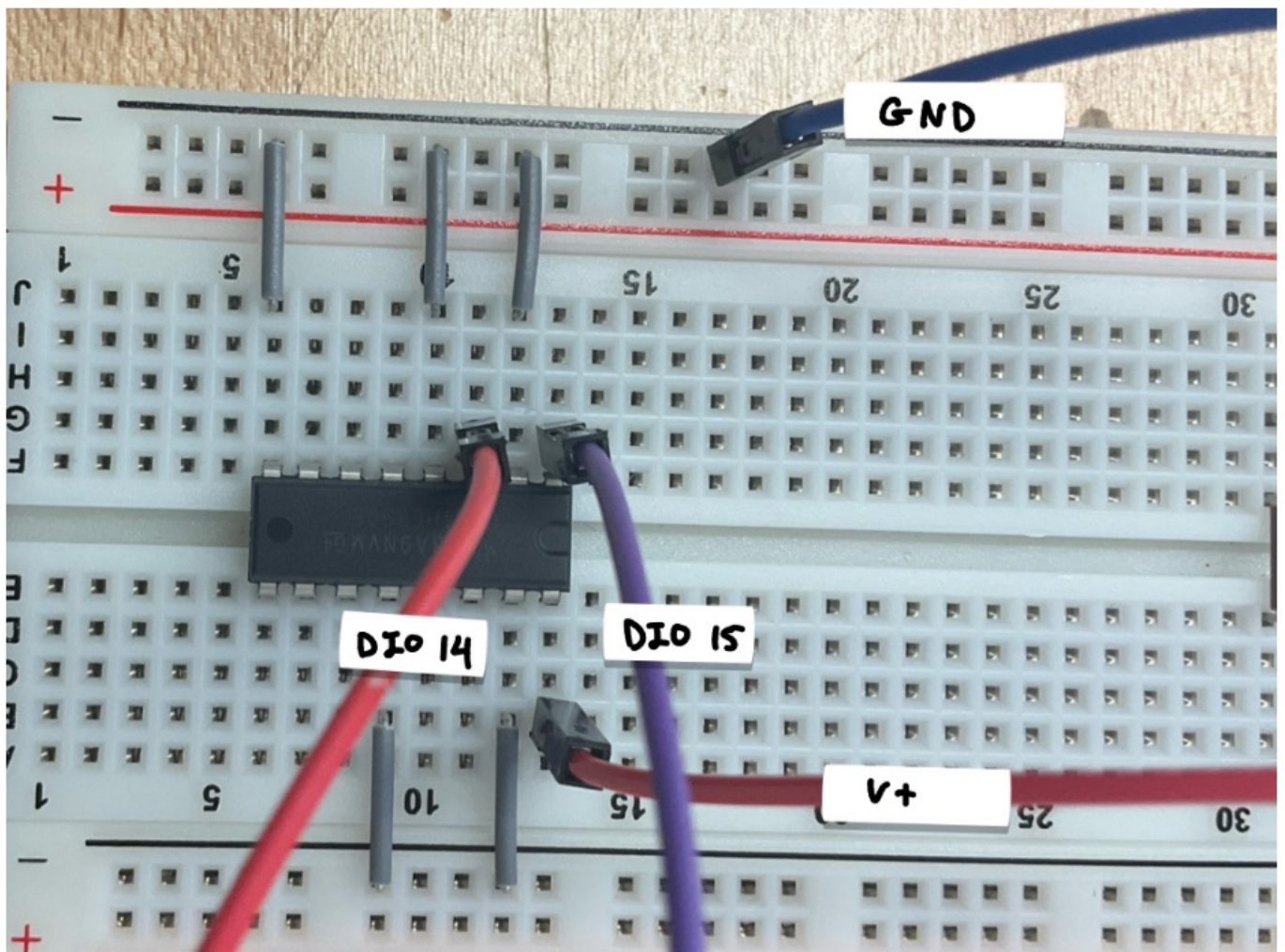


Figure 29, divide by 2 circuit setup.

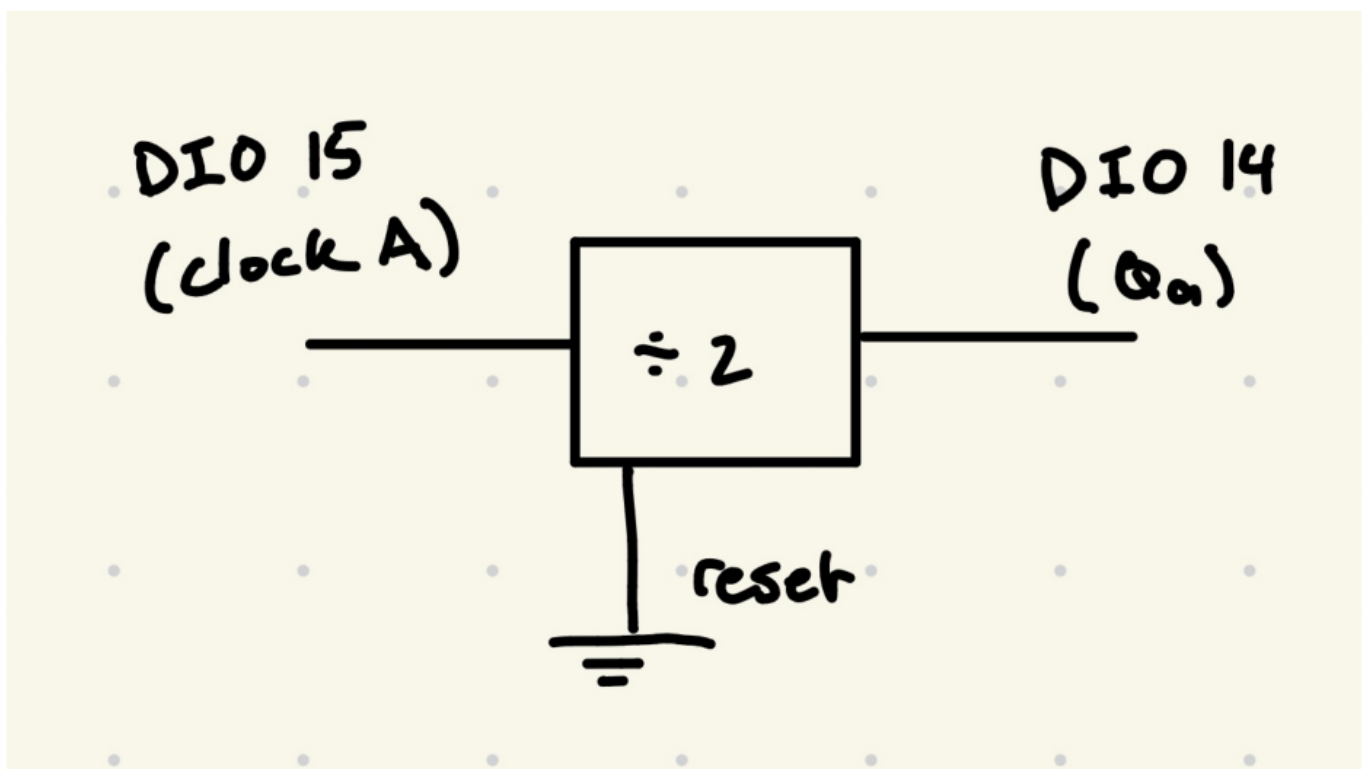


Figure 30, divide by 2 circuit schematic.

I set the input to the counter to be a digital input of 1kHz, and observed the output signal;

In [55]:

```
InlineImage('divideby2.png', width=12, height=12, caption= "Figure 31, AD3 reading of t
```

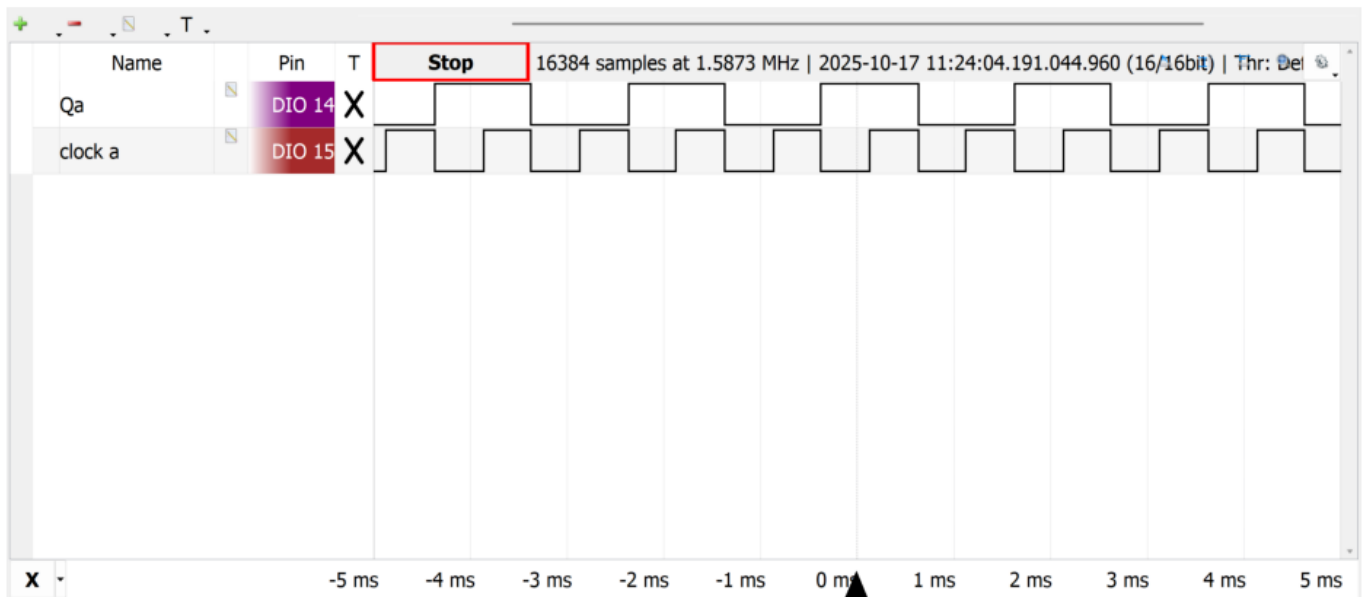


Figure 31, AD3 reading of the counter input and output at a 1kHz digital input.

As we can see, the output Qa toggles at every falling edge it reads from the input to clock a. This is consistent with the data sheet for the counter. Now that the theoretical behaviour for the clock has been verified, I can move on to the next step.

The next step is making a decade counter. I can do this by wiring up the output of the divide by 2 count to the input of the divide by 5 count. If I drive the divide by 2 count, the output will drive the divide by 5 and I should theoretically be able to get a count up to 9, and the 10th count being a reset to 0. Below, I include my circuit diagram, my schematic for the counter, and a photo of my setup.

In [56]:

```
#images here
```

```
InlineImage('divby10photo.png', width=12, height=12, caption= "Figure 32, divide by 10 c
```

```
InlineImage('divby10schema.png', width=12, height=12, caption= "Figure 33, divide by 10
```

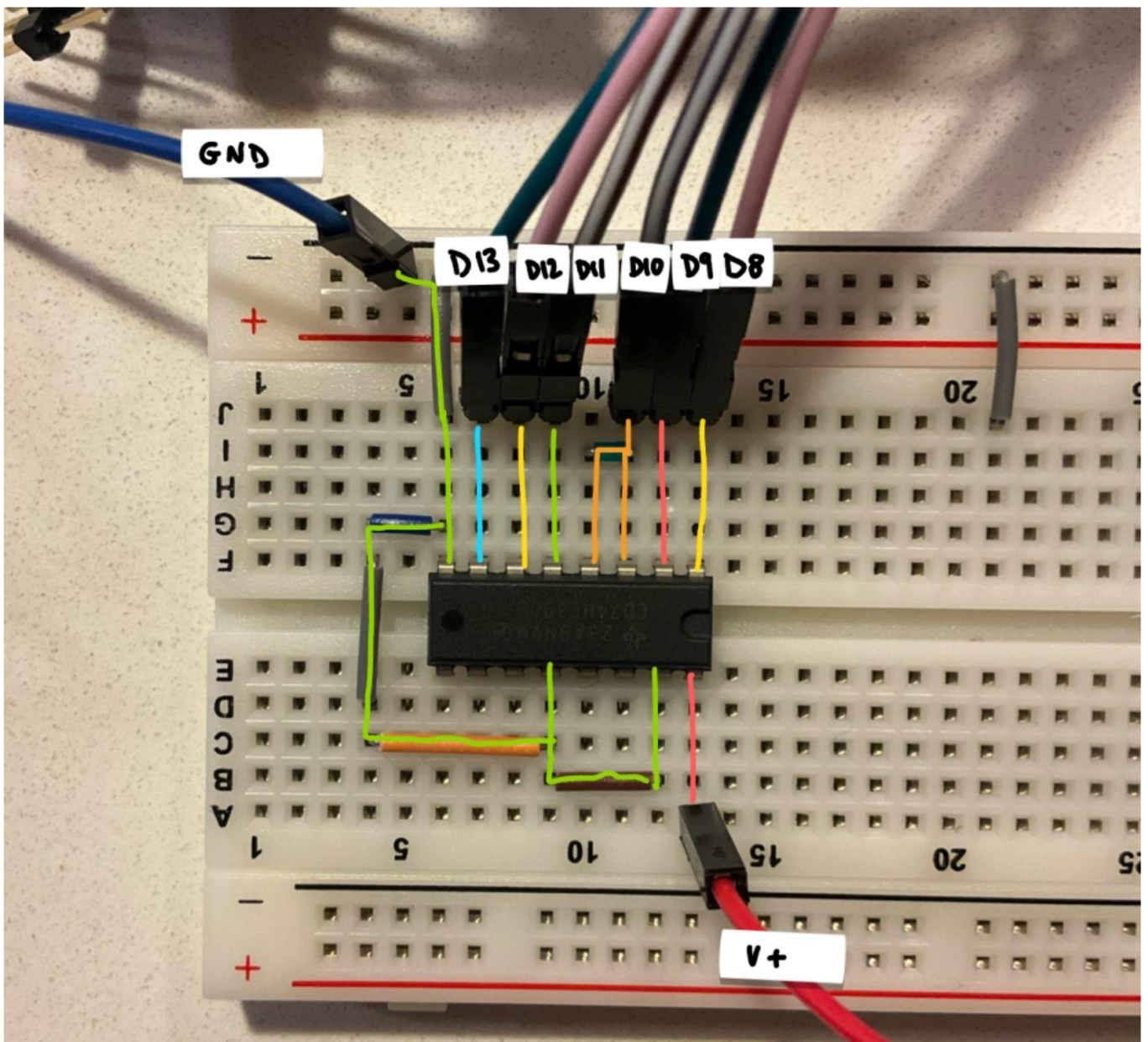


Figure 32, divide by 10 circuit setup.

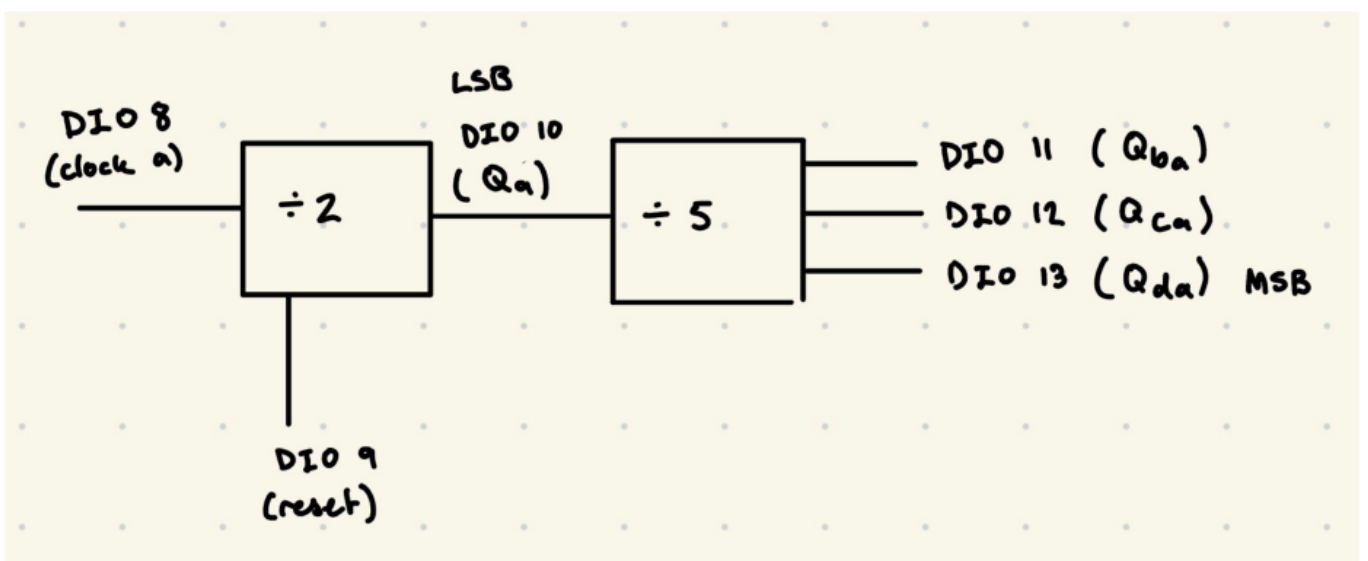


Figure 33, divide by 10 circuit schematic.

Below is what I see using the BUS in the LOGIC app in waveforms, which converts the count from the digital IO pins into a decimal number. I used the BCD bus as my table partner Marnu reminded me, since he said I would not get a correct reading otherwise.

```
In [57]:
InlineImage('divby10.png', width=12, height=12, caption= "Figure 34, AD3 LOGIC software
```

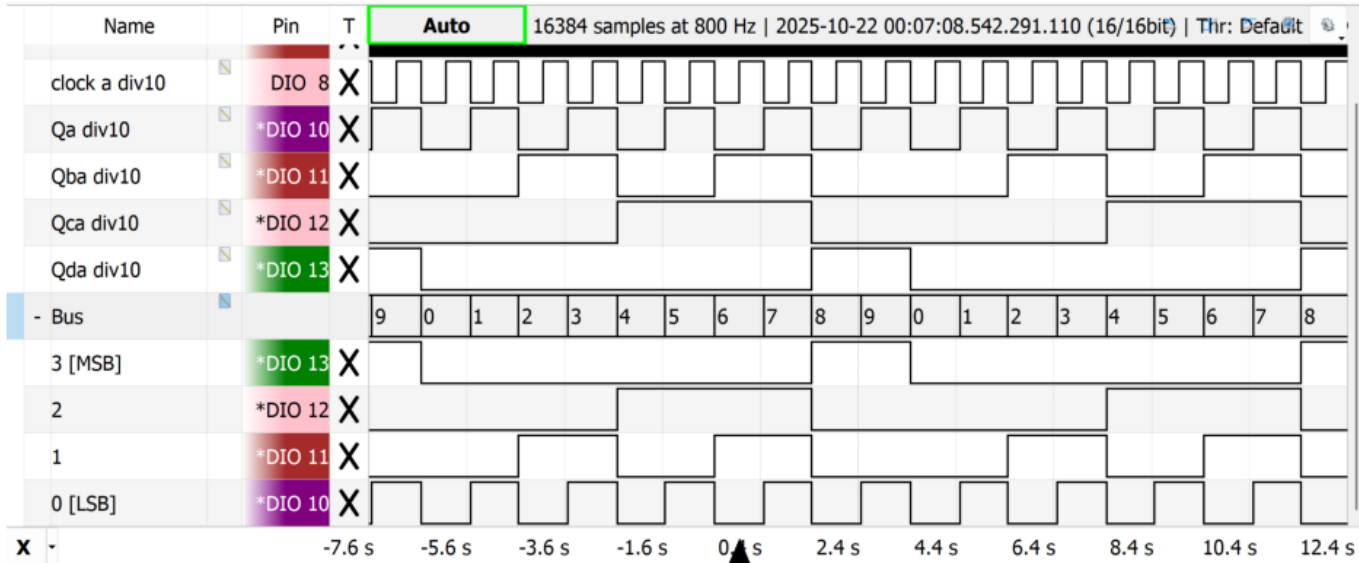
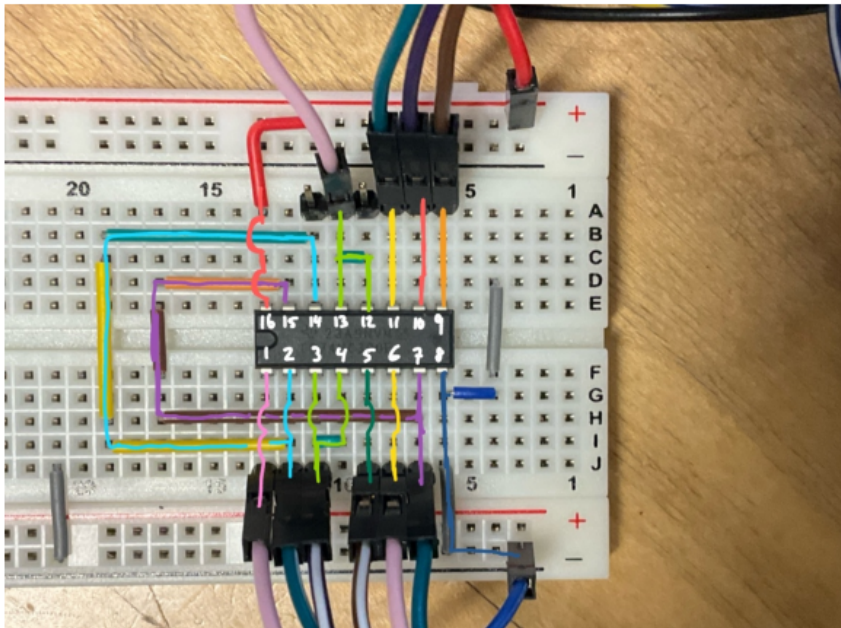


Figure 34, AD3 LOGIC software displaying the IO readings of each input/output in my divide by 10 counter. BUS is set up to read the outputs only.

Now, I will construct a divide by 100 counter. I will do this by utilizing all of the available counters in the MC54/74HC39 ripple counter. I can take my current setup for the divide by 10, and connect the ouput Qda (the most significant bit) to the input of the other divide by 2 counter, and then connect the output of that to the last divide by 5 counter. Then, I can use the BUS app to read the count as a decimal value in the same way as I have done for the decade counter. $2 \times 5 \times 2 \times 5 = 100$.

Below is an image and diagram of this counter setup, as well as the waveforms digital output logic app readings to prove its functionality. Once again, I am using a 1Hz DIO pin to drive the first clock. I only need one input as the output of each clock drives its neighbor's input.

```
In [58]:
InlineImage('divby100annotated.png', width=12, height=12, caption= "Figure 35, annotated
InlineImage('divby100photo.png', width=12, height=12, caption= "Figure 36, divide by 100
InlineImage('divby100schema.png', width=12, height=12, caption= "Figure 37, divide by 10
```

1 = DIO 8	9 = DIO 7
2 = DIO 9	10 = DIO 6
3 = } DIO 10	11 = DIO 5
4 = }	12 = } DIO 0
5 = DIO 11	13 = }
6 = DIO 12	14 = DIO 9
7 = DIO 13	15 = DIO 13
8 = GND	16 = V _{cc}

Figure 35, annotated divide by 100 circuit diagram for self to debug/rebuild.

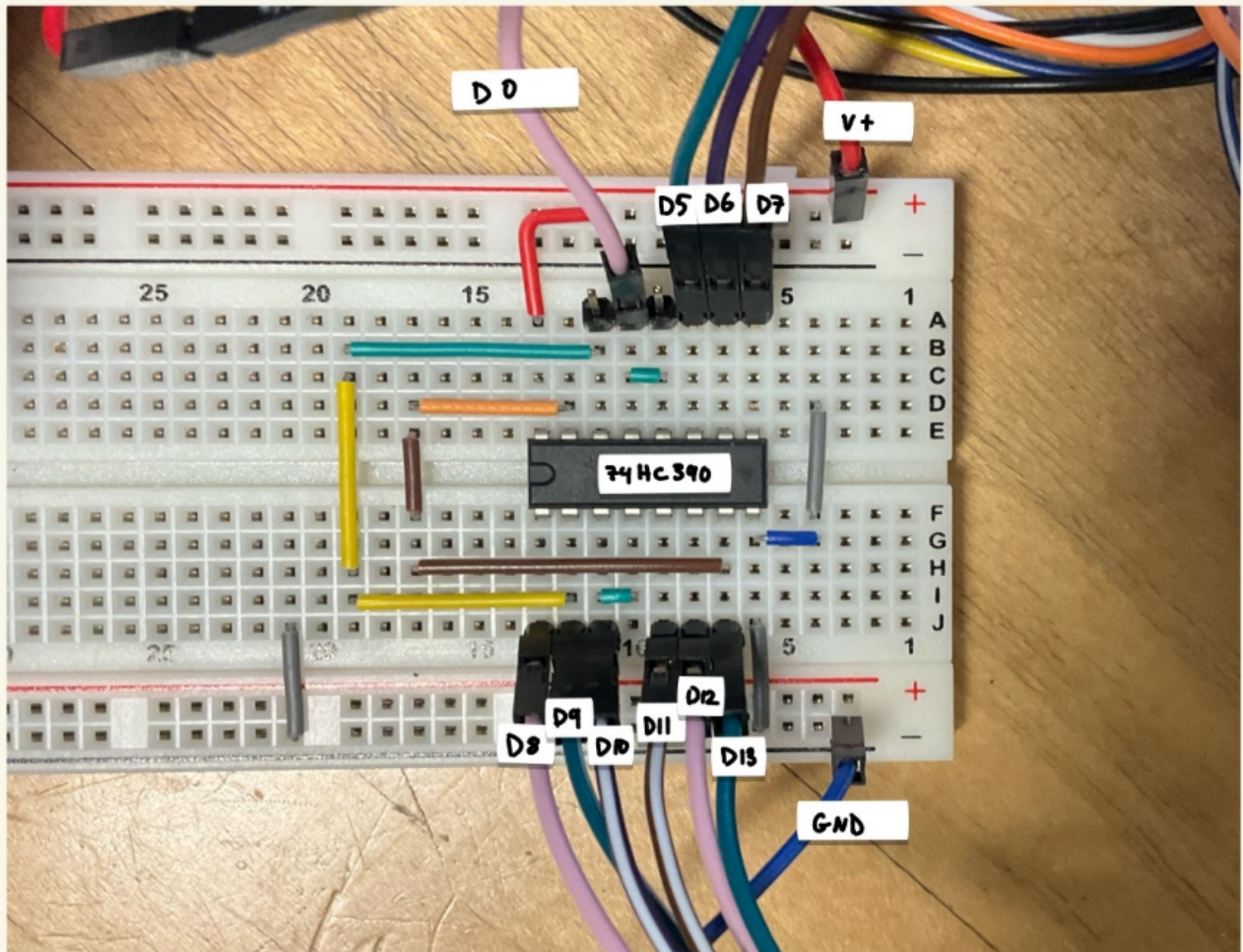


Figure 36, divide by 100 circuit setup.

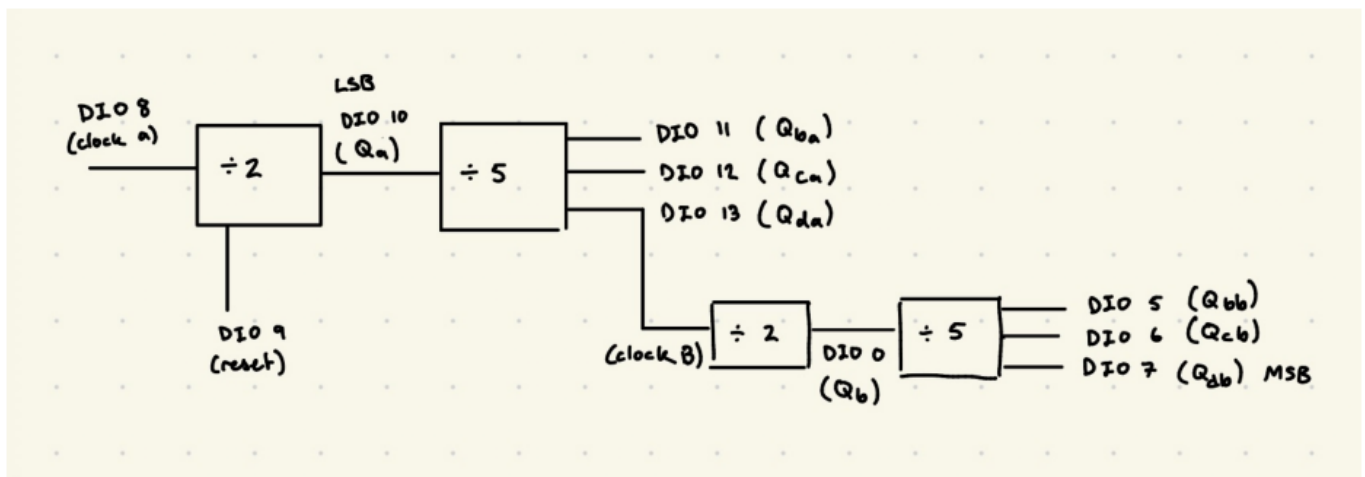


Figure 37, divide by 100 circuit schematic.

In [59]:

InlineImage('COUNT100.png', width=12, height=12, caption= "Figure 38, working divby100 c

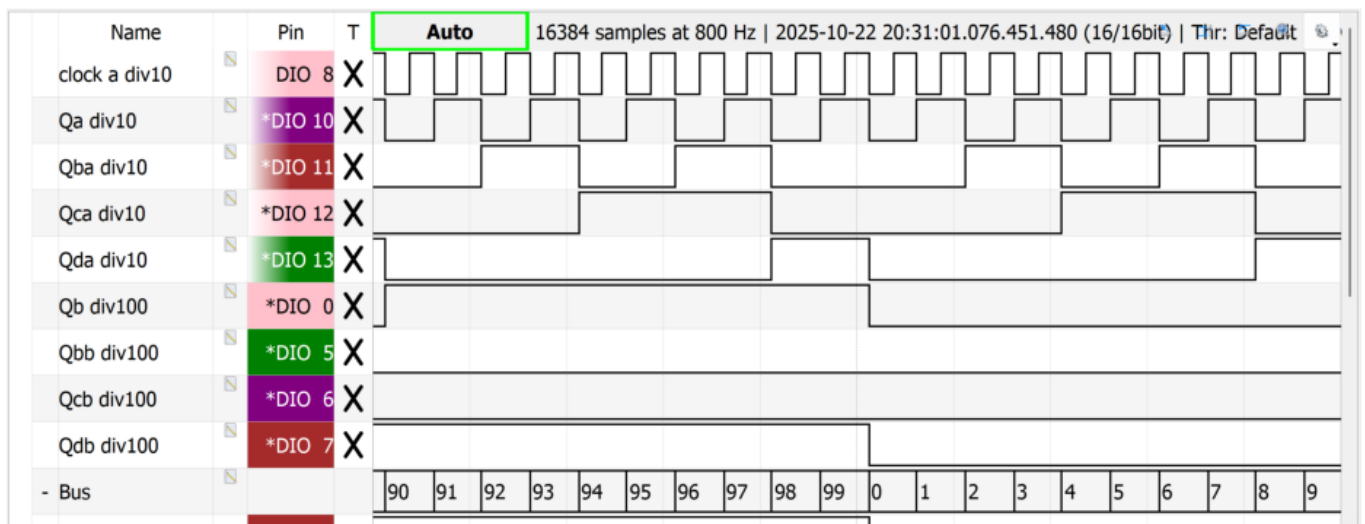


Figure 38, working divby100 counter!!.

#3. Conclusion

In this lab, we explored a lot of things. We looked at an inverter, a DAC, a NAND gate and how it works with a D-latch, and how counters work. We verified the behaviour of inverters and nand gates.

Here is what I learned about the inverter:

- the hysteresis effect is due to the different voltage thresholds to trigger a rise or fall (ie change from low to high or vice versa). This makes it strong against noise.

Here is what I learned about the DAC:

- the dac's resolution is effectively the amplitude of Vcc divided by the number of times the least significant bit triggers for the most significant bit to trigger once.
- the dac is like a counting ladder, similar to an actual counter. However, this provides a more gradual count (recall sawtooth wave) than the on/off of a regular counter.
- the dac can probably be used for a count, but in different applications than a normal counter. Maybe something that requires more gradual in-between states, such as a sweeping sonar thing..?

Here is what I learned about the D-Latch:

- the d latch can store 1 bit of information, and the storing occurs when the E input is high. Once it is low, the stored value/previous Q value is kept.
- The reason for the D-latch value storage is due to its feedback on Q and Qbar.

Here is what I learned about the counters:

- even though there are theoretically 8 'bits' in the system, it does not count to 256 because of the way it is configured. There are only 100 possible states in this system [0-99] so it only counts 100 times, and it resets once it passes 99.
- We can convert a digital input count to a decimal value by using the BCD bus in waveforms software.

To conclude, I did not know how these types of circuits worked prior to this lab. I think that this knowledge will help greatly in the future for robot summer.

Another takeaway is that good diagrams are more important now.

In [2]:

```
# comments in Python start with a "#" and will not be executed - well commented code is
# Importing modules that will be necessary for the tutorial
import numpy as np # numpy is a general math/array manipulation library that is almost a
import matplotlib.image as mimg # matplotlib.image is used for displaying images in py
import matplotlib.pyplot as plt # matplotlib.pyplot is used for general plotting and is
import pandas as pd # pandas is a data manipulation library, we will use it to import da
print('All necessary modules imported!')
```

All necessary modules imported!

In [3]:

```
def InlineImage(img, width=8, height=8, caption=None, fs=24):
    # Author: BG
    # Renders an image with a caption underneath using matplotlib
    # Requires matplotlib.pyplot to be imported as plt and matplotlib.image to be imported
    # img - string, the location of an image file
    # width, height - numbers, the dimensions of the figure the image is displayed in. 0
    # caption - string, the caption to be displayed beneath the image
    # fs - number, the font size of the caption
    fig, ax = plt.subplots(figsize=(width, height))
    ax.imshow(mimg.imread(img))
    ax.set_xlabel(caption, fontsize=fs)
    ax.tick_params(
        which='both',
        bottom=False,
        top=False,
        left=False,
        labelleft=False,
        labelbottom=False)
    ax.spines["top"].set_visible(False)
    ax.spines["left"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["bottom"].set_visible(False)
```